

Informe técnico

Viajes Aventura

Sistema de gestión de destinos, paquetes y reservas

TI3V21 · Programación Orientada a Objeto Seguro

Unidad 4 · Unidad integradora de la competencia

Evaluación Sumativa N.º 4

Sección V-IEI-N2-P3-C1 · INACAP Valparaíso

Docente: Rubén Schnettler

Leonardo Guerrero

05-10-2026

Índice

Introducción	3
1. Problema y alcance	3
1.1 Contexto y problema del cliente	4
1.2 Interesados, alcance y marco legal	5
1.3 Supuestos declarados	7
2. Requerimientos	9
2.1 De las reglas de negocio a los requerimientos	9
2.2 Requerimientos funcionales	10
2.3 Requerimientos no funcionales	14
2.4 Priorización con el método MoSCoW	16
2.5 Criterios de aceptación	17
2.6 Matriz de verificación: problema → requerimiento	20
3. Modelado de la solución	21
3.1 Casos de uso del sistema, con sus actores	21
3.2 Procesos de negocio en BPMN	25
3.3 Diagrama de clases	30
4. Planificación ágil	37
4.1 Roles y responsabilidades	37
4.2 Product Backlog	38
4.3 Sprint Backlog	40
4.4 Cronograma y entregables	44
5. Implementación	45
5.1 Estructura del código y cómo se ejecuta	45
5.2 Del diagrama al código	46
5.3 Persistencia	47
5.4 Operaciones CRUD	48
5.5 Validación en tres capas	48
5.6 Acceso seguro: autenticación	49
5.7 Validación de credenciales	49
5.8 Protección de los datos personales	50
5.9 Evaluación y mejora de la seguridad con apoyo de IA	51
5.10 Pruebas y evidencia	52
6. Uso de inteligencia artificial	52
7. Trazabilidad	54
8. Conclusiones	60
Referencias	60

Introducción

Contexto. Viajes Aventura es una agencia de turismo de Valparaíso con tres socios y sin otro personal. En su última temporada registró 214 reservas sobre 8 a 12 paquetes y un catálogo de 18 destinos, todo en una planilla compartida y un cuaderno. Esa forma de trabajar le costó dinero y clientes: reservas por sobre el cupo de un paquete, paquetes cobrados a un precio distinto del publicado, un paquete vendido después de su fecha de salida, reservas duplicadas y una planilla con el RUT y el teléfono de los clientes que circula por correo. El capítulo 1 documenta cada una de esas fallas con su cifra.

Objetivo general. Diseñar, construir y asegurar un sistema que reemplace la planilla y el cuaderno de la agencia, en Python con base de datos `sqlite3`, con autenticación y con los datos personales de los clientes protegidos, de modo que las fallas de la temporada no puedan repetirse.

Objetivos específicos, uno por cada criterio de la evaluación:

- Identificar, clasificar y priorizar los requerimientos a partir del caso, y verificar que respondan a sus problemas (criterio 4.1.1, capítulos 1 y 2).
- Modelar los actores, los procesos y la estructura de la solución con casos de uso, BPMN y un diagrama de clases UML trazables entre sí (criterio 4.1.2, capítulos 3 y 7).
- Planificar y ejecutar el trabajo con Scrum: roles, Product Backlog, sprints con entregables y tiempos reales (criterio 4.1.3, capítulo 4).
- Implementar el modelo en Python con persistencia y CRUD completos, usando la IA de forma crítica (criterio 4.1.4, capítulos 5 y 6).
- Proteger el acceso y los datos personales, y evaluar la seguridad con criterios técnicos y apoyo de IA (criterio 4.1.5, secciones 5.6 a 5.9).

Alcance del informe. Cubre el sistema que pide el caso (§6): destinos, paquetes, clientes y reservas, con dos roles, en una aplicación de terminal. Lo que el caso deja fuera (pago, facturación, aplicación para teléfonos, integraciones) se nombra como Won't en el capítulo 2. El trabajo es individual, y el capítulo 4 justifica cómo se adaptó Scrum a una persona.

Cómo se lee. Bajo el título de cada sección aparece, en cursiva, el indicador de la rúbrica que responde, con sus palabras, para que la evidencia se encuentre sin buscarla. Cada sección cierra con un párrafo «Esto implica» que dice qué consecuencia tiene lo mostrado. Las tablas y figuras están numeradas y se citan antes de aparecer. El código, las pruebas y la evidencia de la seguridad y del uso de IA están en el repositorio público github.com/leonardoguerrero/P00-Viajes-Aventura: docs/AUDITORIA.md, docs/ANALISIS_IA.md, docs/SALIDA_TERMINAL.md y docs/ia/.

1. Problema y alcance

Este capítulo describe lo que el cliente declaró, en sus términos, antes de decidir nada sobre la solución: el problema con sus cifras (1.1), quiénes participan, qué queda dentro y fuera del alcance y qué datos personales trata el sistema (1.2), y los supuestos que cubren los vacíos del caso (1.3). Los requerimientos que resuelven cada problema están en el capítulo 2.

1.1 Contexto y problema del cliente

Responde a 4.1.1.1.4: verifica que los requerimientos respondan a la problemática planteada por el cliente.

Viajes Aventura es una agencia de Valparaíso con tres socios y sin otro personal. Arma paquetes turísticos combinando destinos que conoce y los vende a personas naturales, que pagan por transferencia. En la última temporada registró 214 reservas sobre 8 a 12 paquetes publicados y un catálogo de 18 destinos. No tiene sistema: el catálogo vive en una planilla compartida, las reservas en un cuaderno y los datos de los clientes en ese cuaderno, en la planilla y en los teléfonos de los socios.

La tabla 1 reúne cada consecuencia verificable que el caso documenta, con su cifra o su cita. Cada una lleva un identificador (P-01 a P-15) que la matriz de la sección 2.6 cruza con los requerimientos que la resuelven.

Tabla 1. Consecuencias del problema documentadas en el caso (P-01 a P-15)

ID	Consecuencia	Evidencia en el caso
P-01	La misma reserva queda escrita dos veces por socios distintos	§4: 19 reservas duplicadas detectadas al cierre · §2: filas 1-2 y 3-4 de la hoja «Reservas temporada»
P-02	Se venden más lugares que el cupo del paquete	§4: 6 reservas por sobre el cupo · §3.2, Matías: «vendí catorce lugares en un paquete que tenía doce»
P-03	Se venden paquetes cuya fecha de salida ya pasó	§4: 3 reservas sobre fechas vencidas · §3.2, Matías: «Estaba en la planilla, se veía disponible, y nadie lo había sacado»
P-04	El precio de un paquete vendido cambia cuando cambia el costo de un destino	§4: 4 paquetes con precio distinto del cobrado · §2: Sofía Larraín, 2.100.000 y 2.475.000 por la misma reserva
P-05	Consultar lo que un cliente reservó exige revisar el cuaderno	§4: 31 consultas respondidas revisando el cuaderno · §3.3, Ignacio: «tengo que revisar el cuaderno hoja por hoja»
P-06	Destinos que ya no se operan siguen ofreciéndose	§4: 4 destinos no operados y visibles · §2
P-07	El volumen de datos no permite responder preguntas con certeza	§4: 214 reservas registradas · §3.3, Ignacio: «no tengo cómo responder con certeza»
P-08	Borrar un destino deja sin contenido a los paquetes ya vendidos	§3.1, Paulina: «Si los borro, esas filas quedan sin nada»
P-09	El precio de un paquete se calcula a mano y se publica con errores	§3.1, Paulina: «me equivoqué en un cero y el paquete salió publicado a la décima parte de lo que costaba»
P-10	El mismo destino aparece repetido con nombres distintos	§2: «Hay destinos repetidos con nombres distintos»
P-11	No se sabe cuándo se actualizó el costo de un destino	§2: «costos que nadie recuerda cuándo se actualizaron por última vez»
P-12	La fórmula del precio se rompe al copiar filas	§2: «una fórmula que a veces se rompe al copiar filas»
P-13	Los datos del cliente están repartidos en tres lugares	§3.3, Ignacio: «están en el cuaderno, en la planilla y en los teléfonos de los tres»
P-14	El RUT y el teléfono circulan sin protección	§3.3, Ignacio: «esa planilla se ha enviado por correo varias veces. Si se pierde, se pierde con todo lo de los clientes adentro»

P-15	La confirmación de una reserva queda solo en la mensajería del socio que atendió	§2: los datos quedan «a veces, solo en la conversación de mensajería del socio que atendió»
-------------	--	---

La guía de la evaluación resume la situación en cinco problemas (pág. 5): falta de control de los destinos disponibles, dificultades en la gestión de paquetes, problemas en el registro y seguimiento de reservas, riesgos en la gestión de datos sensibles y procesos manuales que afectan la eficiencia. Los quince de la tabla son la forma concreta de esos cinco, y la matriz de la sección 2.6 los cruza también.

La causa común. Los quince síntomas tienen un mismo origen: cada dato del negocio existe en varias copias escritas a mano (planilla, cuaderno y mensajería), y ninguna de esas copias aplica las reglas del negocio en el momento en que se escribe. Una reserva duplicada, un cupo sobrevendido y un precio que cambia después de vendido son la misma falla vista desde tres lados: nadie comprueba la regla cuando el dato entra.

Esto implica que el sistema no se diseña como una planilla mejorada, sino como un **registro único** en el que cada regla se comprueba antes de guardar. Esa decisión es la que ordena todo el catálogo que sigue.

1.2 Interesados, alcance y marco legal

Responde a 4.1.1.G.1: identifica y documenta los requerimientos funcionales y no funcionales de la solución.

La tabla 2 identifica a los interesados por su rol frente al sistema.

Tabla 2. Interesados del sistema

Interesado	Rol respecto del sistema	Interés principal	Influencia
Cliente	Persona natural que se registra, reserva y consulta su historial	Reservar con certeza de cupo y de precio, y saber qué reservó	Media
Cliente sin cuenta	La misma persona antes de registrarse, que mira la oferta (supuesto S-09). En los casos de uso es el actor Cliente: el actor es un rol, no una persona	Ver paquetes, fechas y precios antes de registrarse	Baja
Administrador	Uno de los tres socios: mantiene destinos, arma y publica paquetes, fija cupos	Un catálogo sin repetidos, precios calculados sin error y cupos que se respetan	Alta
Socios como dueños	Paulina Ovalle (catálogo), Matías Bórquez (reservas) e Ignacio Salas (datos)	Cobrar el precio publicado y no perder los datos de los clientes	Alta
Agencia de Protección de Datos Personales	No opera el sistema: fiscaliza el tratamiento de los datos personales desde el 1-dic-2026 (Ley 21.719)	Datos mínimos, protegidos y con finalidad declarada	Media

El alcance es el que acordaron los socios (§6 del caso). Lo que queda fuera se nombra, porque es lo que evita discusiones posteriores, y en el catálogo aparece como prioridad *Won't* con la condición bajo la cual entraría (tabla 3).

Tabla 3. Alcance del sistema según el caso (§6)

Dentro del alcance	Fuera del alcance (Won't en esta versión)
Destinos: registrar, modificar, dejar no disponible, eliminar y listar. Paquetes: crear combinando destinos, definir fechas, cupo y margen, calcular el precio, publicar y consultar disponibilidad. Reservas: registro y autenticación de clientes, reservar, almacenar la reserva, consultar el historial propio y anular. Seguridad: autenticación, validación de todo dato ingresado y protección de credenciales y datos sensibles.	Pasarela de pago y verificación automática de transferencias · facturación electrónica ante el SII · aplicación para teléfonos · integración con aerolíneas, hoteles u operadores · envío de correos o mensajería · informes de gestión para los socios · contabilidad y remuneraciones.

Marco legal: qué datos personales trata el sistema

El tratamiento de datos personales en Chile se rige por la Ley 19.628, sobre protección de la vida privada, modificada por la Ley 21.719, que la moderniza, cambia su nombre a «protección de los datos personales» y crea la Agencia de Protección de Datos Personales. La Ley 21.719 se publicó el 13 de diciembre de 2024 y rige desde el 1 de diciembre de 2026, dos meses después de la entrega de este sistema: por eso se diseña desde ya conforme a ella. El informe usa sus definiciones (artículo 2 de la ley modificada):

- **Dato personal** (letra f): «cualquier información vinculada o referida a una persona natural identificada o identificable», y es identificable quien puede determinarse «mediante uno o más identificadores, tales como el nombre, el número de cédula de identidad» u otros elementos de su identidad.
- **Dato personal sensible** (letra g): el que se refiere a «las características físicas o morales de las personas o a hechos o circunstancias de su vida privada o intimidad, que revelen el origen étnico o racial, la afiliación política, sindical o gremial, la situación socioeconómica, las convicciones ideológicas o filosóficas, las creencias religiosas, los datos relativos a la salud, al perfil biológico humano, los datos biométricos, y la información relativa a la vida sexual, a la orientación sexual y a la identidad de género de una persona natural».

Con esas definiciones, el RUT y el teléfono que la regla R17 llama «datos sensibles» son, en términos legales, **datos personales**: el número de cédula es justamente el identificador que la ley pone de ejemplo. R17 usa «sensible» en el sentido del negocio (datos cuya filtración perjudica al cliente), y esa protección se mantiene completa; lo que cambia es el fundamento, que no es el régimen reforzado de los datos sensibles sino los principios de **seguridad** y **confidencialidad** (artículo 3) y el **deber de adoptar medidas de seguridad** del artículo 14 quinquies, que nombra expresamente «la seudonimización y el cifrado de datos personales» y exige asegurar su «confidencialidad, integridad, disponibilidad y resiliencia». En este informe, «información sensible» (la expresión de la rúbrica) abarca las credenciales y los datos personales que R17 ordena resguardar (tabla 4).

Tabla 4. Datos personales que trata el sistema y su protección

Dato	Clasificación legal	Finalidad (por qué se pide)	Protección exigida (requerimiento)
Nombre	Dato personal (identificador, letra f)	Identificar al titular de la cuenta y de la reserva (R9, R12)	Visible para el propio cliente y para el administrador en las reservas de un paquete
RUT	Dato personal; la ley nombra la cédula como identificador	Lo exige el registro del cliente (R9)	Cifrado en la base (RNF-SEG-02) y enmascarado en pantalla (RF-SEG-10)
Teléfono	Dato personal	Contactar al cliente por su reserva (R9; §3.2)	Cifrado en la base (RNF-SEG-02) y enmascarado en pantalla (RF-SEG-10)
Correo	Dato personal	Identificar la cuenta e iniciar sesión (R9)	Único en la base; nunca aparece en mensajes de error (RF-SEG-02)
Contraseña	Credencial: no se guarda como dato (R10)	Autenticar al usuario (R11)	Solo un resumen que no se puede revertir, nunca la contraseña (RNF-SEG-01)
Historial de reservas	Dato personal, vinculado al titular	Consultar lo reservado (R11)	Solo lo ve su titular y, por paquete, el administrador (RF-SEG-05)
Datos personales sensibles (letra g)	No se tratan	Ninguna finalidad del caso los necesita	El sistema no los pide: principio de proporcionalidad (artículo 3)

En todo el informe se usan las definiciones legales de la Ley 19.628 modificada por la Ley 21.719 (texto oficial en LeyChile, BCN, normas 141599 y 1209272, consultado el 30-sep-2026). «Dato sensible» se reserva para la letra g) del artículo 2; al RUT y al teléfono se los llama «datos personales que R17 ordena resguardar». Las citas del caso y de la guía que dicen «datos sensibles» se dejan literales.

La «situación socioeconómica» está entre los datos sensibles de la letra g). Los montos que un cliente paga por sus reservas son datos de una transacción, no una declaración de su situación económica, y el sistema no los usa para perfilar a nadie; por eso se tratan como dato personal, visible solo para su titular y para los socios.

1.3 Supuestos declarados

Responde a 4.1.1.l.4. El caso pide adoptar un supuesto donde falte información, declararlo y fundamentarlo técnicamente.

Los tres primeros cubren los vacíos que el propio caso nombra (§6: el desistimiento, el fin de temporada y quién modifica el catálogo). Los demás salieron al escribir los requerimientos, cuando una regla admitía más de una lectura (tabla 5).

Tabla 5. Supuestos declarados

ID	Supuesto adoptado	Fundamento	Si resulta falso
----	-------------------	------------	------------------

S-01	Un cliente puede anular una reserva vigente hasta el día anterior a la salida. La reserva no se borra: queda «anulada» y sus personas vuelven al cupo. La devolución del dinero ocurre fuera del sistema	Es el vacío «qué ocurre cuando un cliente desiste». Conservar la fila mantiene el historial (R11) y el total cobrado (R13); borrarla repetiría el problema del cuaderno (P-05)	Si la agencia no acepta anulaciones, se oculta la opción; el modelo no cambia
S-02	Un paquete deja de ofrecerse solo, cuando llega su fecha de salida. Su estado se calcula desde la fecha, no desde una acción manual, y el paquete se conserva para el historial	Es el vacío «cómo se comporta un paquete cuando su temporada termina». Matías vendió un paquete vencido porque «nadie lo había sacado» (P-03): una solución que dependa de sacarlo a mano repite la falla	Si quieren retirarlo antes, se agrega la acción «retirar» sin tocar esta regla
S-03	Solo el rol administrador modifica el catálogo y los paquetes, y cada socio tiene su propia cuenta de administrador	Es el vacío «quién puede modificar el catálogo». El caso asigna esa tarea al administrador (§1.3). Una cuenta por socio permite saber quién hizo cada cambio, cosa que la planilla compartida no permite	Si un cliente pudiera editar algo, cambia la matriz de permisos (RF-SEG-05)
S-04	La primera cuenta de administrador se crea en el primer uso, cuando la base no tiene usuarios. No existen credenciales escritas en el código	Evita las contraseñas por defecto, una causa conocida de fallas de autenticación (OWASP A07)	Ninguno: es una decisión de seguridad, no del negocio
S-05	El margen es un porcentaje sobre la suma de los costos base, con 20 % como valor propuesto, y el precio se redondea al peso entero	R6 dice «más ese margen» y fija el 20 % habitual; los costos del caso están en pesos enteros (§1.1)	Si fuera un monto fijo, cambia solo la fórmula de RF-PAQ-04
S-06	Se puede reservar hasta el día anterior a la salida; el mismo día de salida ya no	R15 prohíbe reservar si la salida «ya pasó». El día de salida el grupo parte, y el cliente no alcanza a llegar al punto de encuentro (Matías, §3.2)	Se cambia una comparación en una sola regla
S-07	Un paquete en borrador se edita completo. Uno publicado conserva destinos, fechas y precio; solo su cupo puede cambiar, y nunca por debajo de las personas ya reservadas	R7 fija el precio al publicar. Cambiar fechas o destinos después de vender afectaría a clientes que ya compraron pasajes (§3.2)	Si se permitiera cambiar fechas, habría que avisar a los clientes, y eso está fuera del alcance
S-08	La reserva registra si está «vigente» o «anulada», no si está pagada. El pago se sigue verificando a mano	La verificación de transferencias está fuera del alcance (§6)	La versión 2 agrega el estado de pago (W-01)
S-09	Cualquier persona, sin cuenta, puede ver los paquetes disponibles; reservar y ver reservas exige iniciar sesión	R11 restringe solo reservar y consultar reservas. Ver la oferta es el paso previo a registrarse	Se exige sesión también para ver la oferta; cambia un permiso
S-10	Una reserva de varias personas la hace un cliente titular; no se piden datos de los acompañantes	R12 registra solo la cantidad de personas. Pedir menos datos personales es el principio de minimización de la ley	Se agrega una tabla de pasajeros, con los mismos resguardos que el RUT

S-11	El sistema se usa en un equipo de la oficina, en terminal, con una base SQLite local. Los tres socios entran con su cuenta y puede haber más de una sesión abierta a la vez	Tres personas sin personal de informática; la aplicación para teléfonos está fuera del alcance (§6)	El acceso remoto exige un servidor y cambiar la capa de acceso, no el dominio
S-12	La clave que cifra el RUT y el teléfono se genera en el primer uso y se guarda fuera de la base y de la carpeta del sistema (en la carpeta de configuración del usuario), con permisos solo del dueño. Se respalda aparte, porque si se pierde esos datos no se recuperan	Separar la clave de los datos es lo que hace que una copia de la base enviada por correo no exponga a nadie (P-14)	Si no se respalda la clave, perderla deja ilegibles los RUT; se declara como riesgo
S-13	El correo identifica al cliente. El RUT se valida, pero no se exige que sea único	R9: «El correo identifica al cliente y no se repite»	Si se exigiera RUT único, se agrega un índice ciego (resumen HMAC) sin descifrar
S-14	El sistema parte sin datos: las reservas y los destinos de la planilla y del cuaderno no se migran. Así no puede existir una reserva por sobre el cupo heredada de la temporada anterior	La planilla trae 19 reservas duplicadas, 6 sobre el cupo y 4 precios distintos de lo cobrado (§4): migrarla tal cual metería esos errores en el sistema que debe impedirlos. El alcance (§6) no menciona migración	Se agrega una carga inicial que pase cada fila por las mismas validaciones (R1 a R17) y deje en un informe aparte las que no cumplen, para que un socio las resuelva a mano
S-15	Un destino no disponible desaparece de la oferta para armar paquetes nuevos, pero sigue a la vista, con su nombre, dentro de los paquetes que ya lo contienen	R8: «deja de ofrecerse para paquetes nuevos, de modo que los paquetes ya vendidos conserven su contenido». El cliente compró ese contenido y tiene que poder verlo en su historial	Si los socios prefieren ocultarlo también en los paquetes vigentes, el listado de paquetes filtra los destinos por estado; el modelo no cambia
S-16	Dentro del sistema, nadie ve completos el RUT ni el teléfono de otra persona. El administrador contacta al cliente por su correo, que aparece en el listado de reservas de cada paquete; el cliente ve sus propios datos enmascarados	R17 declara sensibles solo el RUT y el teléfono, no el correo. Ignacio teme que la planilla circule por correo (§3.3): mientras menos personas vean un dato, menos puede filtrarse (minimización, Ley 21.719)	Si un socio necesita llamar al cliente, se agrega una acción «ver teléfono» solo para el administrador, que deje registro de quién lo vio y cuándo

2. Requerimientos

Este capítulo traduce el caso en requerimientos del sistema. El caso advierte que sus diecisiete reglas «no son requerimientos: una regla describe cómo funciona el negocio y un requerimiento describe qué debe hacer el sistema». Por eso el orden es: la traducción de cada regla (2.1), el catálogo agrupado, numerado y priorizado con el método MoSCoW (2.2 a 2.4), los criterios que permiten comprobar cada requerimiento obligatorio (2.5) y la matriz que verifica que cada problema del capítulo 1 quedó cubierto (2.6). Aquí no se nombran clases ni tablas: eso es diseño, y está en el capítulo 3.

2.1 De las reglas de negocio a los requerimientos

Responde a 4.1.1.G.3 (trazabilidad) y a 4.1.1.I.4.

El caso pide traducir cada regla en lo que el sistema debe hacer. La tabla muestra esa traducción para las diecisiete reglas: ninguna quedó sin un requerimiento que la haga cumplir (tabla 6).

Tabla 6. Traducción de las reglas de negocio R1 a R17

Regla	Qué le exige al sistema	Requerimientos
R1	Pedir los cinco datos del destino y rechazar un nombre repetido	RF-DES-01, RF-DES-02, RNF-FIA-01
R2	Rechazar un costo base menor o igual a cero	RF-DES-03, RNF-FIA-01
R3	Rechazar paquetes con menos de 2 o más de 5 destinos, o con un destino repetido	RF-PAQ-02, RNF-FIA-01
R4	Permitir que un destino esté en varios paquetes a la vez	RF-PAQ-01
R5	Pedir nombre, fechas y cupo, con regreso posterior a la salida y cupo mayor que cero	RF-PAQ-01, RF-PAQ-03, RNF-FIA-01
R6	Rechazar un margen negativo y calcular el precio por persona	RF-PAQ-03, RF-PAQ-04
R7	Fijar el precio al publicar y no cambiarlo después	RF-PAQ-05, RF-PAQ-08
R8	Eliminar el destino sin paquetes; marcar no disponible el que tiene, y no ofrecerlo en paquetes nuevos	RF-DES-06, RF-DES-07, RF-PAQ-02
R9	Registrar al cliente con sus cinco datos y rechazar un correo repetido	RF-RES-01, RF-RES-02, RF-RES-03
R10	Guardar la contraseña solo como resumen criptográfico	RNF-SEG-01
R11	Exigir sesión para reservar y consultar, y mostrar a cada cliente solo lo suyo	RF-SEG-01, RF-SEG-05, RF-RES-04, RF-RES-08
R12	Asociar la reserva a un cliente y un paquete, con fecha, personas y total	RF-RES-04, RF-RES-07
R13	Calcular el total al reservar y no cambiarlo nunca	RF-RES-07
R14	Llevar el cupo disponible y rechazar lo que lo supere, incluso con dos reservas simultáneas	RF-PAQ-06, RF-RES-05, RNF-FIA-02
R15	Comparar la fecha de salida contra la fecha del día	RF-PAQ-07, RF-RES-06
R16	Rechazar una reserva de cero personas	RF-RES-05, RNF-FIA-01
R17	No mostrar RUT ni teléfono en listados ni en errores, y protegerlos donde se guardan	RF-SEG-10, RF-SEG-13, RNF-SEG-02, RNF-SEG-05

Esto implica que las reglas no se programan como avisos en el menú: cada una tiene un requerimiento verificable, y las que protegen la integridad del dato (R1, R2, R3, R5, R9 y R16) se exigen también en la base de datos (RNF-FIA-01), para que un error en el código no alcance a guardar un dato inválido.

2.2 Requerimientos funcionales

Responde a 4.1.1.G.1 (identificación) y 4.1.1.G.3 (redacción).

Los requerimientos se **agrupan** por módulo del alcance, se **numeran** con el prefijo de su grupo y se **priorizan** con el método MoSCoW (criterio en la sección 2.4). Todos siguen la misma estructura: «El sistema debe...» seguido de una acción observable, un solo verbo principal y ningún adjetivo que no

se pueda medir. La columna «Origen» cita la regla, la sección, la entrevista o el supuesto de donde sale cada uno, y «Depende de» indica qué debe existir antes.

Grupo DES · Destinos (actor: Administrador)

Los requerimientos de este grupo están en la tabla 7.

Tabla 7. Requerimientos funcionales del grupo Destinos

ID	Requerimiento	Origen	Prioridad	Depende de
RF-DES-01	El sistema debe permitir al administrador registrar un destino con nombre, zona, descripción, duración en días y costo base por persona.	R1 · §6 · guía pág. 5	Must	RF-SEG-01
RF-DES-02	El sistema debe rechazar un destino cuyo nombre ya exista en el catálogo, comparando sin distinguir mayúsculas, tildes ni espacios sobrantes.	R1 · P-10	Must	RF-DES-01
RF-DES-03	El sistema debe rechazar un destino con costo base menor o igual a cero o con duración menor a un día.	R2 · R1	Must	RF-DES-01
RF-DES-04	El sistema debe permitir al administrador modificar el nombre, la zona, la descripción, la duración y el costo base de un destino, con las mismas validaciones del registro.	§6 · guía pág. 5	Must	RF-DES-01
RF-DES-05	El sistema debe registrar la fecha de la última modificación del costo base de cada destino.	P-11	Should	RF-DES-04
RF-DES-06	El sistema debe eliminar un destino solo cuando no forma parte de ningún paquete.	R8 · guía pág. 5	Must	RF-DES-01
RF-DES-07	El sistema debe marcar como no disponible, en lugar de eliminar, un destino que forma parte de al menos un paquete, y conservar el contenido de esos paquetes.	R8 · P-06 · P-08	Must	RF-DES-06
RF-DES-08	El sistema debe listar al administrador los destinos del catálogo con su estado, zona, duración, costo base y fecha de actualización del costo.	§6 · guía pág. 5 · P-06	Must	RF-DES-01
RF-DES-09	El sistema debe permitir al administrador volver a marcar como disponible un destino no disponible.	R8 (un destino puede volver a operarse)	Could	RF-DES-07
RF-DES-10	El sistema debe permitir al administrador filtrar el listado del catálogo para ver solo los destinos disponibles.	§6 («listar destinos disponibles») · P-06	Must	RF-DES-08

Grupo PAQ · Paquetes (actores: Administrador; Cliente en la consulta)

Los requerimientos de este grupo están en la tabla 8.

Tabla 8. Requerimientos funcionales del grupo Paquetes

ID	Requerimiento	Origen	Prioridad	Depende de
----	---------------	--------	-----------	------------

RF-PAQ-01	El sistema debe permitir al administrador crear un paquete con nombre, fecha de salida, fecha de regreso, cupo máximo, margen de operación y los destinos que combina, pudiendo un mismo destino estar en varios paquetes.	R4 · R5 · R6 · §1.2 · guía pág. 5	Must	RF-DES-01
RF-PAQ-02	El sistema debe rechazar un paquete con menos de dos o más de cinco destinos, con un destino repetido o con un destino no disponible.	R3 · R8	Must	RF-PAQ-01
RF-PAQ-03	El sistema debe rechazar un paquete cuya fecha de regreso no sea posterior a la de salida, cuyo cupo no sea mayor que cero o cuyo margen sea negativo.	R5 · R6 · S-05	Must	RF-PAQ-01
RF-PAQ-04	El sistema debe calcular el precio por persona como la suma de los costos base de los destinos más el margen, redondeado al peso, y mostrarlo con el costo de cada destino antes de guardar.	R6 · P-09 · P-12 · guía pág. 5	Must	RF-PAQ-01
RF-PAQ-05	El sistema debe permitir al administrador publicar un paquete y fijar, al publicarlo, su precio por persona, que no cambia aunque después cambie el costo de un destino.	R7 · P-04	Must	RF-PAQ-04
RF-PAQ-06	El sistema debe mostrar los paquetes disponibles con su nombre, fechas, destinos, precio por persona y cupo disponible, calculado como el cupo máximo menos las personas de las reservas vigentes.	R14 · §6 · guía pág. 5 · P-02 · S-09	Must	RF-PAQ-05
RF-PAQ-07	El sistema debe excluir de los paquetes disponibles, sin eliminarlos, los paquetes cuya fecha de salida ya llegó.	R15 · P-03 · S-02 · S-06	Must	RF-PAQ-06
RF-PAQ-08	El sistema debe permitir al administrador modificar todos los datos de un paquete en borrador, y en uno publicado solo el cupo máximo, sin dejarlo por debajo de las personas ya reservadas.	R7 · R14 · S-07	Should	RF-PAQ-05
RF-PAQ-09	El sistema debe permitir al administrador eliminar un paquete solo cuando no tiene reservas.	R12 · S-01	Should	RF-PAQ-01
RF-PAQ-10	El sistema debe listar al administrador todos los paquetes con su estado (borrador, publicado o vencido) y las personas reservadas en cada uno.	§1.3 · P-02	Should	RF-PAQ-06
RF-PAQ-11	El sistema debe proponer un margen de 20 % cuando el administrador no indica uno.	R6 · S-05	Should	RF-PAQ-03

Grupo RES · Clientes y reservas (actor: Cliente)

Los requerimientos de este grupo están en la tabla 9.

Tabla 9. Requerimientos funcionales del grupo Clientes y reservas

ID	Requerimiento	Origen	Prioridad	Depende de
RF-RES-01	El sistema debe permitir a una persona registrarse como cliente con nombre, RUT, correo electrónico, teléfono y contraseña.	R9 · §6 · guía pág. 5 · P-13	Must	RF-SEG-04
RF-RES-02	El sistema debe rechazar el registro de un correo que ya pertenece a otra cuenta.	R9 · P-13	Must	RF-RES-01

RF-RES-03	El sistema debe validar, antes de guardar, el dígito verificador del RUT, el formato del correo y el formato del teléfono chileno: nueve dígitos que empiezan entre 2 y 9, con o sin el prefijo +56.	R9 · §6 (validación de todo dato)	Must	RF-RES-01
RF-RES-04	El sistema debe permitir a un cliente con sesión iniciada reservar un paquete disponible indicando la cantidad de personas.	R11 · R12 · §6 · guía pág. 5 · P-01 · P-15	Must	RF-SEG-01, RF-PAQ-06
RF-RES-05	El sistema debe rechazar una reserva de menos de una persona o de más personas que el cupo disponible del paquete en ese momento.	R14 · R16 · P-02	Must	RF-RES-04
RF-RES-06	El sistema debe rechazar una reserva sobre un paquete cuya fecha de salida ya llegó.	R15 · P-03 · S-06	Must	RF-RES-04
RF-RES-07	El sistema debe guardar con cada reserva su fecha de emisión y su total, calculado como el precio fijado del paquete por la cantidad de personas, sin volver a calcularlo después.	R12 · R13 · P-04	Must	RF-RES-04, RF-PAQ-05
RF-RES-08	El sistema debe mostrar a cada cliente solo su propio historial de reservas, completo, con paquete, fechas, personas, total y estado.	R11 · §6 · guía pág. 6 · P-05	Must	RF-RES-04
RF-RES-09	El sistema debe permitir al cliente anular una reserva propia vigente antes del día de salida, dejándola en estado anulada y devolviendo sus personas al cupo.	S-01	Should	RF-RES-08
RF-RES-10	El sistema debe advertir al cliente y pedir su confirmación cuando ya tiene una reserva vigente en el mismo paquete.	P-01	Should	RF-RES-04
RF-RES-11	El sistema debe mostrar al administrador las reservas de un paquete con el nombre y el correo de cada cliente, las personas, el total y el estado.	§1.3 · P-02 · R17	Should	RF-RES-04
RF-RES-12	El sistema debe permitir al cliente actualizar su nombre y su teléfono.	R9	Could	RF-RES-01

Grupo SEG · Acceso y seguridad (actores: Cliente y Administrador)

Los requerimientos de este grupo están en la tabla 10.

Tabla 10. Requerimientos funcionales del grupo Acceso y seguridad

ID	Requerimiento	Origen	Prioridad	Depende de
RF-SEG-01	El sistema debe permitir iniciar sesión con correo y contraseña.	R11 · §6 · guía pág. 6	Must	RF-SEG-06
RF-SEG-02	El sistema debe responder con el mismo mensaje cuando el correo no existe y cuando la contraseña es incorrecta, sin indicar cuál de los dos falló.	§6 (protección de credenciales) · guía pág. 6	Must	RF-SEG-01
RF-SEG-03	El sistema debe bloquear el inicio de sesión de una cuenta durante 5 minutos después de 5 intentos fallidos consecutivos.	§6 · guía pág. 6 · P-14	Must	RF-SEG-01

RF-SEG-04	El sistema debe rechazar una contraseña de menos de 12 caracteres o igual al correo de la cuenta.	R10 · §6 (validación de todo dato)	Must	(ninguna)
RF-SEG-05	El sistema debe permitir solo al administrador gestionar destinos y paquetes, solo al cliente reservar, y a cada cliente acceder únicamente a sus reservas.	R11 · S-03	Must	RF-SEG-01
RF-SEG-06	El sistema debe crear en el primer uso la cuenta del primer administrador.	S-03 · S-04	Must	RF-SEG-04
RF-SEG-07	El sistema debe permitir a un administrador crear cuentas de administrador para los demás socios.	§1 (tres socios) · S-03	Should	RF-SEG-06
RF-SEG-08	El sistema debe cerrar la sesión cuando el usuario lo pide.	§6 (autenticación)	Must	RF-SEG-01
RF-SEG-09	El sistema debe cerrar la sesión después de 10 minutos sin actividad.	S-11 (equipo compartido de oficina)	Should	RF-SEG-01
RF-SEG-10	El sistema debe mostrar el RUT y el teléfono enmascarados en todo listado.	R17 · P-14	Must	RF-RES-01
RF-SEG-11	El sistema debe permitir a un usuario cambiar su contraseña después de ingresar la actual.	§6 (protección de credenciales)	Should	RF-SEG-04
RF-SEG-12	El sistema debe crear solo cuentas de cliente desde el registro público.	S-03 · S-04	Must	RF-RES-01
RF-SEG-13	El sistema debe excluir el RUT y el teléfono de todo mensaje de error.	R17 · P-14	Must	RF-RES-01

2.3 Requerimientos no funcionales

Responde a 4.1.1.G.1 y 4.1.1.G.3.

Se agrupan por característica de calidad de la norma ISO/IEC 25010 (seguridad, fiabilidad, rendimiento, usabilidad, mantenibilidad y portabilidad), más el cumplimiento legal. Cada uno tiene una condición que se puede medir o comprobar, porque un requerimiento no funcional sin número no se puede aceptar ni rechazar (tabla 11).

Tabla 11. Requerimientos no funcionales por categoría

ID	Categoría	Requerimiento y condición medible	Origen	Prioridad
RNF-SEG-01	Seguridad	El sistema debe guardar las contraseñas solo como resumen Argon2id con sal única, generado con la librería <code>argon2-cffi</code> del repositorio oficial PyPI. Se comprueba leyendo la base: ninguna contraseña aparece en claro y todo resumen empieza con <code>\$argon2id\$</code> .	R10 · guía pág. 8	Must
RNF-SEG-02	Seguridad	El sistema debe guardar el RUT y el teléfono cifrados con cifrado autenticado (Fernet: AES más HMAC-SHA256), con la clave fuera del código y de la base. Se comprueba leyendo la base, donde no aparece ningún RUT legible, y alterando un byte del dato cifrado, que debe producir un error de integridad y no un dato falso.	R17 · P-14 · S-12 · Ley 21.719 art. 14 quinquies	Must

RNF-SEG-03	Seguridad	El sistema debe validar tipo, largo y formato de todo dato ingresado dentro de la clase que lo guarda, y ejecutar el SQL solo con parámetros. Se comprueba con cero consultas armadas por concatenación (bandit, regla B608).	§6 · guía pág. 6	Must
RNF-SEG-04	Seguridad	El sistema debe dejar el archivo de la base y el de la clave con permisos de lectura y escritura solo para su dueño (0600) en Linux y macOS. Se comprueba leyendo los permisos de los dos archivos después del primer uso.	P-14	Should
RNF-SEG-05	Seguridad	El sistema debe responder a todo error con un mensaje para el usuario, sin trazas internas, rutas de archivos ni datos personales. Se comprueba con cero «Traceback» en la sesión de prueba completa.	R17	Must
RNF-FIA-01	Fiabilidad	La base de datos debe rechazar por sí misma los datos que violan R1, R2, R3, R5, R9 y R16, con restricciones CHECK, UNIQUE y claves foráneas. Se comprueba insertando por SQL directo un dato inválido de cada regla, que debe ser rechazado.	P-12 · R1 · R2 · R3 · R5 · R9 · R16	Must
RNF-FIA-02	Fiabilidad	El sistema debe comprobar el cupo y registrar la reserva en una misma transacción, de modo que dos reservas simultáneas nunca sumen más personas que el cupo. Se comprueba con dos conexiones que reservan a la vez el último lugar: solo una debe tener éxito.	R14 · P-02	Must
RNF-FIA-03	Fiabilidad	El sistema debe permitir al administrador generar un respaldo de la base que se pueda abrir y que contenga todos los registros, sin detener a las demás sesiones; la clave de cifrado se respalda aparte. Se comprueba abriendo el respaldo y contando sus registros.	P-14 · S-12	Should
RNF-REN-01	Rendimiento	Con diez veces el volumen de una temporada (2.140 reservas, 180 destinos y 120 paquetes), listar los paquetes disponibles o el historial de un cliente debe tardar menos de 1 segundo.	§1 · P-07	Should
RNF-REN-02	Rendimiento	La verificación de una contraseña debe tardar entre 0,1 y 1 segundo en el equipo de la oficina: costosa para un ataque de fuerza bruta y tolerable para el usuario.	RNF-SEG-01 · S-11	Should
RNF-USA-01	Usabilidad	El usuario debe poder cancelar cualquier operación escribiendo «x» en cualquier dato, sin que se guarde nada parcial, y cada error debe decir qué dato corregir.	§1 (socios sin personal técnico)	Should
RNF-USA-02	Usabilidad	El sistema debe mostrar los montos en pesos con separador de miles (1.050.000) y las fechas como día-mes-año, igual que la planilla actual de la agencia. Se comprueba en la sesión del menú: montos como \$1.056.000 y fechas como 02-11-2026.	§2	Should

RNF-USA-03	Usabilidad	El menú debe mostrar solo las opciones que el rol de la sesión puede usar. Se comprueba en la sesión del menú: el cliente no ve ninguna opción de destinos, paquetes ni cuentas.	RF-SEG-05	Should
RNF-MAN-01	Mantenibilidad	El código debe implementar el diagrama de clases sin diferencias, y ninguna sentencia SQL debe quedar fuera de las clases del dominio. Se comprueba con un script que compara diagrama y código.	guía pág. 7 (Paso 4)	Must
RNF-MAN-02	Mantenibilidad	Cada regla R1 a R17 debe tener una prueba automática que se ejecute en cada envío al repositorio. Se comprueba con el workflow del repositorio, que falla si alguna regla deja de cumplirse.	R1 a R17	Should
RNF-POR-01	Portabilidad	El sistema debe funcionar en Windows, macOS y Linux con Python 3.12 o superior, sin servidor de base de datos, instalando solo dependencias de versión fijada y verificadas por su resumen (hash).	§3.3 (computadores personales) · S-11	Must
RNF-LEG-01	Cumplimiento	El sistema debe tratar los datos personales según la Ley 19.628 modificada por la Ley 21.719: pedir solo los de R9 (proporcionalidad), usarlos solo para registrar, reservar y contactar (finalidad), protegerlos con las medidas del artículo 14 quinquies y no tratar datos personales sensibles (artículo 2, letra g). Se comprueba con la tabla del marco legal (1.2): cada dato con su finalidad y su protección.	R9 · R17 · P-14 · S-10 · Ley 21.719	Must

2.4 Priorización con el método MoSCoW

Responde a 4.1.1.G.2: clasifica y prioriza los requerimientos de acuerdo con las necesidades del proyecto.

La clasificación tiene dos ejes. El primero es el **tipo**: funcional (qué hace el sistema, 2.2) o no funcional (con qué calidad lo hace, 2.3). El segundo es la **prioridad**, con el método MoSCoW. El criterio de cada letra se fijó antes de asignarlas, para que la prioridad no dependa de la opinión del momento (tabla 12):

Tabla 12. Criterio y conteo de cada prioridad MoSCoW

Prioridad	Criterio de asignación	Funcionales	No funcionales
Must (debe)	Sin él, el sistema viola una regla confirmada (R1 a R17), deja sin resolver una consecuencia cuantificada en §4, no cubre un ítem del alcance de §6 o incumple la ley chilena de datos personales. Si falta, la versión no se entrega.	33	9
Should (debería)	Reduce un problema citado en el caso, pero la agencia tiene una forma manual de suplirlo mientras tanto.	11	8
Could (podría)	Mejora deseable sin un problema cuantificado detrás. Se construye solo si queda tiempo en el sprint.	2	0
Won't (no esta vez)	Fuera del alcance acordado con los socios (§6). Se nombra con la condición bajo la cual entraría.	8	

La tabla 13 reúne los requerimientos Won't, tomados de lo que el caso deja fuera del alcance.

Tabla 13. Requerimientos Won't: lo que queda fuera de esta versión

ID	Queda fuera de esta versión	Condición para que entre
W-01	Pasarela de pago y verificación automática de transferencias	Cuando el volumen haga inviable verificar a mano; exige agregar el estado de pago (S-08)
W-02	Facturación electrónica ante el SII	Cuando la agencia deba emitir boletas desde el sistema
W-03	Aplicación para teléfonos	Cuando el sistema funcione en un servidor (S-11)
W-04	Integración con aerolíneas, hoteles u operadores	Cuando la agencia revenda servicios de terceros, cosa que hoy no hace (§1)
W-05	Envío de correos o mensajería al cliente	Cuando se quiera confirmar la reserva por escrito sin mensajería personal (P-15)
W-06	Informes de gestión para los socios, como cuántas personas han viajado	Versión 2: los datos ya quedan en un registro único (P-07, P-13), así que el informe es una consulta
W-07	Contabilidad y remuneraciones	Fuera del propósito del sistema
W-08	Que el cliente solicite desde el sistema la eliminación de sus datos	Antes del 1-dic-2026 , cuando rige el derecho de supresión de la Ley 21.719. Mientras tanto se atiende a mano

Las dependencias de la columna «Depende de» fijan el orden de construcción dentro de cada prioridad: primero la cuenta y la sesión (RF-SEG-06, RF-SEG-01), después el catálogo (RF-DES-01), los paquetes (RF-PAQ-01 a RF-PAQ-05) y recién entonces las reservas (RF-RES-04 en adelante). Ese orden es el que sigue el Product Backlog del capítulo 4.

Esto implica que los 42 requerimientos Must son el alcance mínimo construible: son los que hacen cumplir las diecisiete reglas, resuelven las seis cifras del §4 y cumplen la ley de datos personales. Todo lo demás se puede postergar sin que el sistema repita los errores de la temporada pasada.

2.5 Criterios de aceptación

Responde a 4.1.1.G.3: un requerimiento está bien redactado cuando se puede comprobar.

Cada requerimiento funcional, Must, Should o Could, tiene un criterio con la forma «dado... cuando... entonces...», que incluye lo que **no** debe ocurrir: esa última parte es la que lo convierte en una prueba. Estos criterios son los que después verifican las pruebas del repositorio (capítulo 5) (tabla 14).

Tabla 14. Criterios de aceptación de los requerimientos funcionales

ID	Criterio de aceptación
RF-DES-01	Dado un administrador con sesión, cuando registra un destino con los cinco datos válidos, entonces el destino aparece en el catálogo como disponible. No debe guardarse si falta cualquiera de los cinco datos.
RF-DES-02	Dado un catálogo con «Valle del Elqui», cuando se registra «valle del elqui» o «Valle del Elquí», entonces se rechaza con el aviso de nombre repetido. No debe quedar un segundo destino.
RF-DES-03	Cuando se registra un destino con costo 0, costo -1 o duración 0, entonces se rechaza indicando el dato. No debe guardarse, ni siquiera insertando por SQL directo (RNF-FIA-01).

RF-DES-04	Dado un destino existente, cuando el administrador cambia su costo a 130.000, entonces el catálogo muestra 130.000. No debe aceptarse un cambio que viole RF-DES-02 o RF-DES-03.
RF-DES-06	Dado un destino que no está en ningún paquete, cuando el administrador lo elimina, entonces desaparece del catálogo. No debe eliminarse si está en algún paquete.
RF-DES-07	Dado un destino incluido en un paquete, cuando el administrador pide eliminarlo, entonces queda «no disponible» y el paquete conserva sus destinos. No debe ofrecerse al crear un paquete nuevo.
RF-DES-08	Cuando el administrador lista el catálogo, entonces ve cada destino con estado, zona, duración, costo y fecha del costo.
RF-DES-10	Dado un catálogo con un destino no disponible, cuando el administrador aplica el filtro de disponibles, entonces ese destino no debe aparecer.
RF-PAQ-01	Dado un catálogo con destinos disponibles, cuando el administrador crea un paquete con datos válidos y dos destinos, entonces el paquete queda en borrador con esos destinos. Un destino que ya está en otro paquete debe poder elegirse (R4).
RF-PAQ-02	Cuando se crea un paquete con uno o seis destinos, con un destino dos veces o con un destino no disponible, entonces se rechaza. No debe guardarse el paquete a medias.
RF-PAQ-03	Cuando el regreso es igual o anterior a la salida, el cupo es 0 o el margen es -5 %, entonces se rechaza indicando el dato.
RF-PAQ-04	Dado un paquete con Salar de Surire (310.000) y Valle del Elqui (120.000) y margen 20 %, entonces el precio por persona es 516.000 y se muestra el costo de cada destino. No debe depender de que alguien sume a mano.
RF-PAQ-05	Dado un paquete publicado a 516.000, cuando el costo del Valle del Elqui sube a 150.000, entonces el paquete sigue en 516.000. Un paquete en borrador, en cambio, debe recalcularse.
RF-PAQ-06	Dado un paquete de cupo 12 con reservas vigentes por 10 personas y una anulada por 2, cuando se consulta la oferta, entonces muestra cupo disponible 2. La reserva anulada no debe descontar cupo.
RF-PAQ-07	Dado un paquete cuya fecha de salida es hoy o una fecha pasada, cuando se consulta la oferta, entonces no aparece. No debe haberse borrado: sigue en el historial y en el listado del administrador.
RF-RES-01	Cuando una persona se registra con los cinco datos válidos, entonces queda una cuenta de cliente con la que puede iniciar sesión. La contraseña no debe quedar legible en ninguna parte (RNF-SEG-01).
RF-RES-02	Dado un cliente con correo carolina@correo.cl, cuando otra persona se registra con Carolina@Correo.cl, entonces se rechaza. El mensaje no debe revelar datos de la cuenta existente.
RF-RES-03	Cuando se registra un RUT con dígito verificador incorrecto, un correo sin arroba o un teléfono con letras, entonces se rechaza indicando qué dato corregir. El mensaje no debe repetir el RUT ingresado (R17).
RF-RES-04	Dado un cliente con sesión y un paquete disponible, cuando reserva para 2 personas, entonces la reserva queda vigente, asociada a él y al paquete. Sin sesión, la opción de reservarse no debe estar disponible.
RF-RES-05	Dado un paquete con cupo disponible 2, cuando un cliente pide 3 personas, entonces se rechaza informando que quedan 2 lugares. Una reserva de 0 personas también debe rechazarse.
RF-RES-06	Dado un paquete cuya salida es hoy, cuando un cliente intenta reservarlo por cualquier vía, entonces se rechaza. No debe bastar con que el paquete esté oculto de la oferta: la regla se comprueba al reservar.
RF-RES-07	Dado un paquete publicado a 516.000, cuando se reserva para 2 personas, entonces la reserva guarda 1.032.000 y la fecha del día. Si después cambia el costo de un destino, el total no debe cambiar.

RF-RES-08	Dados dos clientes con reservas, cuando uno consulta su historial, entonces ve todas las suyas, incluidas las pasadas y las anuladas. No debe ver ninguna reserva del otro cliente.
RF-SEG-01	Dada una cuenta de cliente, cuando inicia sesión con su correo y contraseña, entonces ve el menú de cliente. No debe ver ninguna opción de administrador.
RF-SEG-02	Cuando se intenta entrar con un correo inexistente y con una contraseña errónea de un correo existente, entonces ambos casos reciben el mismo mensaje. No debe poder deducirse qué correos están registrados.
RF-SEG-03	Dada una cuenta con 5 intentos fallidos seguidos, cuando se intenta un sexto con la contraseña correcta, entonces se rechaza hasta que pasen 5 minutos. No debe reiniciarse el bloqueo al cerrar el programa.
RF-SEG-04	Cuando se elige una contraseña de 11 caracteres o igual al correo, entonces se rechaza. Una de 12 o más caracteres distinta del correo debe aceptarse.
RF-SEG-05	Dado un cliente con sesión, cuando intenta una operación del administrador por cualquier vía, incluso llamando al método del dominio, entonces se rechaza por permiso. No debe depender solo de que el menú oculte la opción.
RF-SEG-06	Dada una base sin usuarios, cuando se abre el sistema, entonces pide crear el primer administrador.
RF-SEG-12	Dada una base con usuarios, cuando una persona se registra desde el menú público, entonces la cuenta creada es de cliente. No debe existir forma de elegir el rol administrador.
RF-SEG-08	Cuando el usuario elige cerrar sesión, entonces vuelve a la pantalla de inicio. Ninguna operación que exija sesión debe quedar disponible.
RF-SEG-10	Cuando el administrador lista reservas o clientes, entonces el RUT se ve como 12.***.***-5 y el teléfono como +56 9 **** 1234.
RF-SEG-13	Cuando el registro falla con un RUT o un teléfono inválido, entonces el mensaje nombra el campo. No debe contener el valor ingresado.
RF-DES-05	Dado un destino con costo 120.000, cuando el administrador lo cambia a 130.000, entonces el catálogo muestra la fecha del cambio. No debe cambiar la fecha si el cambio se rechaza.
RF-DES-09	Dado un destino no disponible, cuando el administrador lo vuelve a ofrecer, entonces aparece en el filtro de disponibles y se puede elegir para un paquete nuevo.
RF-PAQ-08	Dado un paquete en borrador, cuando se cambian su nombre, fechas, margen o destinos, entonces se guardan. Dado uno publicado, solo cambia el cupo, y nunca por debajo de las personas reservadas: con 10 reservadas, un cupo de 9 se rechaza.
RF-PAQ-09	Dado un paquete sin reservas, cuando el administrador lo elimina, entonces desaparece del listado. Con una reserva, aunque esté anulada, no debe eliminarse.
RF-PAQ-10	Cuando el administrador lista los paquetes, entonces ve cada uno con su estado (borrador, publicado o vencido) y su cupo disponible frente al máximo.
RF-PAQ-11	Cuando el administrador crea un paquete sin indicar el margen, entonces el margen guardado es 20 %.
RF-RES-09	Dada una reserva vigente propia de un paquete que sale después de hoy, cuando el cliente la anula, entonces queda anulada y sus personas vuelven al cupo. No debe poder anular una reserva ajena ni una de un paquete cuya salida ya llegó.
RF-RES-10	Dado un cliente con una reserva vigente en un paquete, cuando intenta reservar el mismo paquete, entonces se le pide confirmar. Si responde que no, no debe guardarse nada.
RF-RES-11	Cuando el administrador consulta las reservas de un paquete, entonces ve el nombre, el correo, las personas, el total y el estado de cada una. No debe ver ningún RUT ni teléfono.

RF-RES-12	Cuando el cliente cambia su nombre y su teléfono, entonces sus datos muestran los valores nuevos, con el teléfono enmascarado. Un teléfono con letras debe rechazarse.
RF-SEG-07	Dado un administrador con sesión, cuando crea la cuenta de un socio, entonces el socio puede iniciar sesión con su correo y su contraseña. La cuenta creada no debe tener permisos hasta que el socio inicie su sesión.
RF-SEG-09	Dada una sesión sin actividad durante más de 10 minutos, cuando el usuario responde una pregunta o elige una opción, entonces la sesión se cierra. La acción pedida no debe ejecutarse.
RF-SEG-11	Cuando el usuario cambia su contraseña escribiendo la actual, entonces la nueva sirve para entrar. No debe aceptarse con la actual equivocada, igual a la actual ni desde una sesión vieja después de otro cambio.

Los ejemplos numéricos de estos criterios (516.000 por persona, 1.032.000 por dos) salen de los costos del caso (§1.1), y son los mismos datos que usan las pruebas del sistema: si cambia un criterio, cambia su prueba.

2.6 Matriz de verificación: problema → requerimiento

Responde a 4.1.1.1.4: verifica que los requerimientos respondan efectivamente a la problemática planteada por el cliente.

La matriz se lee en los dos sentidos. De izquierda a derecha, comprueba que ningún problema del cliente quedó sin requerimiento. En sentido contrario, que ningún requerimiento Must existe sin un problema, una regla o un ítem del alcance que lo justifique, porque construir lo que nadie pidió también es un error de análisis (tabla 15).

Tabla 15. Matriz de verificación problema → requerimiento

Problema	Cómo lo resuelve el sistema	Requerimientos
P-01	Un registro único: la reserva se escribe una vez, la hace el propio cliente con su cuenta y se advierte si repite paquete	RF-RES-04, RF-RES-10, RF-SEG-05
P-02	El cupo disponible es un dato que el sistema calcula y comprueba en la misma transacción que la reserva	RF-PAQ-06, RF-RES-05, RNF-FIA-02
P-03	La fecha de salida se compara con la fecha del día, al mostrar la oferta y otra vez al reservar	RF-PAQ-07, RF-RES-06
P-04	El precio se fija al publicar y el total se guarda al reservar; un cambio de costo posterior no los alcanza	RF-PAQ-05, RF-RES-07
P-05	El historial del cliente es una consulta, no una búsqueda hoja por hoja	RF-RES-08
P-06	El destino que no se opera queda no disponible y deja de ofrecerse	RF-DES-07, RF-DES-08, RF-DES-10, RF-PAQ-02
P-07	El historial de cada cliente y la lista de quiénes viajan en cada paquete se consultan en menos de un segundo, aun con diez veces el volumen. La pregunta de cuántas personas viajaron en la temporada queda sin responder en esta versión: es el informe de gestión W-06, declarado con su condición de entrada	RF-RES-08, RF-RES-11, RNF-REN-01, W-06
P-08	Borrar solo lo que no está en ningún paquete; lo demás se marca	RF-DES-06, RF-DES-07
P-09	El precio lo calcula el sistema con el desglose a la vista, y un costo mal escrito se rechaza	RF-PAQ-04, RF-DES-03
P-10	El nombre se compara normalizado antes de guardar	RF-DES-02

P-11	Cada cambio de costo guarda su fecha	RF-DES-05
P-12	No hay fórmula que copiar: el cálculo está en el código y las reglas en la base	RF-PAQ-04, RNF-FIA-01
P-13	Una sola cuenta por cliente, identificada por su correo	RF-RES-01, RF-RES-02
P-14	RUT y teléfono cifrados, enmascarados en pantalla y detrás de una sesión con bloqueo	RNF-SEG-02, RNF-SEG-04, RF-SEG-10, RF-SEG-13, RF-SEG-03
P-15	La reserva y su confirmación quedan en el sistema, a la vista del cliente	RF-RES-04, RF-RES-08
G-01	Guía: falta de control de los destinos disponibles (P-06, P-10, P-11)	RF-DES-02, RF-DES-05, RF-DES-07, RF-DES-08
G-02	Guía: dificultades en la gestión de paquetes (P-04, P-09, P-12)	RF-PAQ-04, RF-PAQ-05
G-03	Guía: problemas en el registro y seguimiento de reservas (P-01, P-02, P-03, P-05)	RF-RES-04, RF-RES-05, RF-RES-06, RF-RES-08
G-04	Guía: riesgos en la gestión de datos sensibles de clientes (P-14; en términos legales, datos personales: ver 1.2)	RNF-SEG-01, RNF-SEG-02, RF-SEG-10, RNF-LEG-01
G-05	Guía: procesos manuales que afectan la eficiencia (P-09, P-12, P-15)	RF-PAQ-04, RF-RES-04, RF-RES-08

Esto implica que las seis cifras del §4 que describen fallas tienen al menos un requerimiento Must que las impide, y que las tres entrevistas quedaron cubiertas. La comprobación no es a ojo: un script de trabajo (`trazabilidad.py`, que acompaña a este informe y no viaja al repositorio) lee este informe y falla si una regla, un problema o un requerimiento Must queda sin su pareja.

3. Modelado de la solución

Este capítulo representa la solución con tres modelos que miran el mismo sistema desde lados distintos: los casos de uso muestran qué hace cada actor con el sistema (3.1), los procesos BPMN muestran en qué orden ocurre el trabajo y dónde decide el sistema (3.2), y el diagrama de clases muestra de qué piezas está hecho el software (3.3). Cada diagrama va con su lectura escrita: qué pregunta responde, qué debe notar el lector y qué alternativa se descartó. El capítulo 7 comprueba que los tres describen el mismo sistema que pidió el capítulo 2.

3.1 Casos de uso del sistema, con sus actores

Responde a 4.1.2.G.6: desarrolla diagramas de casos de uso que representen la interacción entre usuarios y sistema.

El diagrama de casos de uso (figura 1) responde quién usa el sistema y para qué. Cada elipse es un objetivo completo que un actor logra: algo después de lo cual podría irse satisfecho. Por eso «validar la fecha» no aparece como caso de uso: es un paso dentro de «Reservar paquete». Los actores son los dos roles del caso (§1.3): Cliente y Administrador. Un actor es un rol, no una persona, así que quien todavía no tiene cuenta y mira la oferta (supuesto S-09) es el mismo actor Cliente.

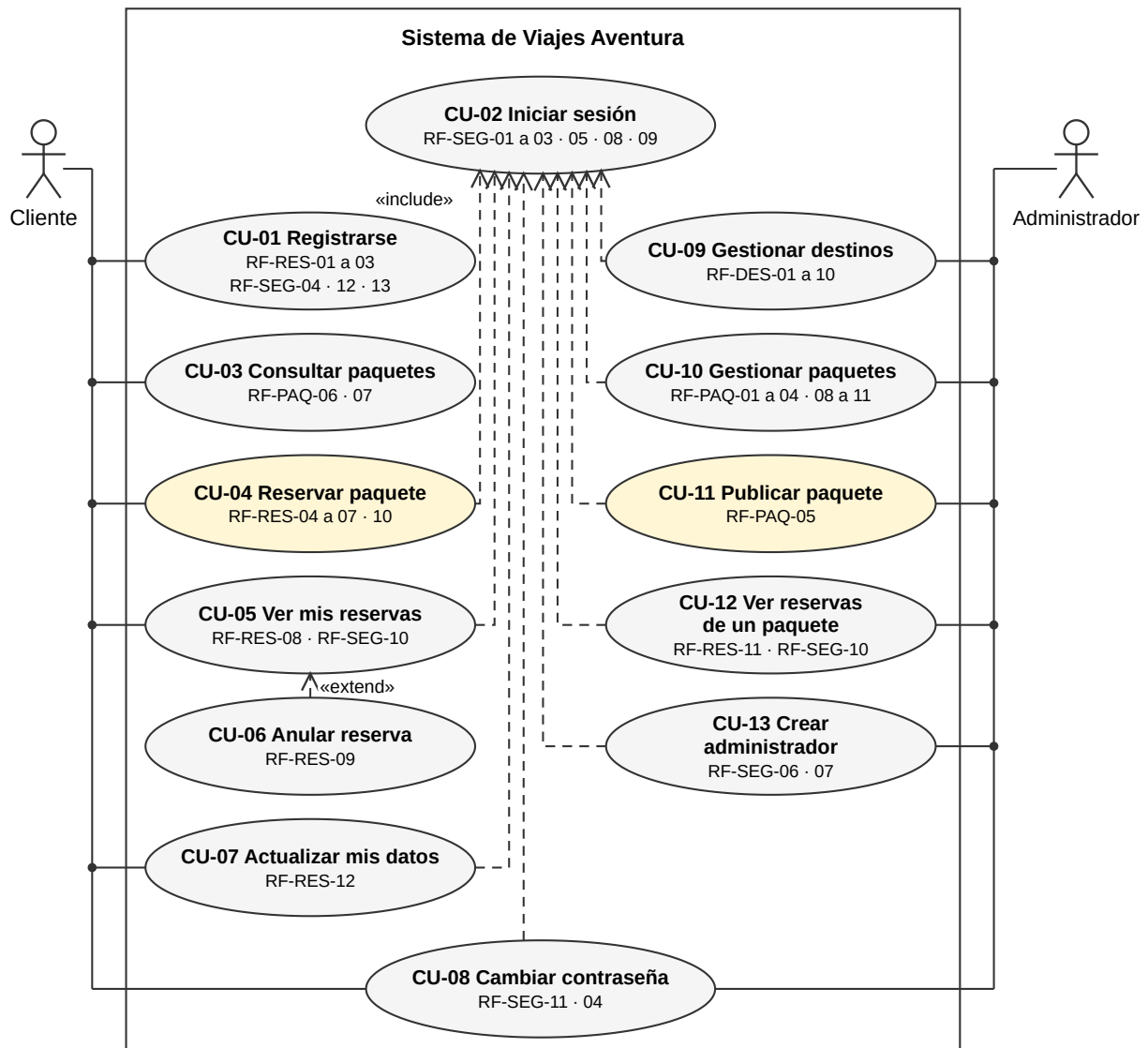


Figura 1. Casos de uso del sistema: trece casos, dos actores y los requerimientos que cumple cada uno.

La figura 1 se lee de arriba hacia abajo en el orden en que ocurren las cosas: el cliente se registra, mira los paquetes, reserva y revisa lo reservado; el administrador mantiene los destinos, arma los paquetes, los publica y revisa quién viaja. Los dos casos en amarillo, «Reservar paquete» y «Publicar paquete», son los que fijan el total y el precio (R13 y R7), donde la agencia perdió dinero en la última temporada. Todos los casos que exigen sesión **incluyen** «Iniciar sesión», que se dibuja una sola vez arriba al centro; cada caso que lo incluye tiene su propia dependencia punteada con su punta, como pide UML 2.5.1, y cada una sube por su carril para que ninguna cruce a otra. «Registrarse» y «Consultar paquetes» no la incluyen, porque se hacen antes de tener sesión. «Anular reserva» **extiende** a «Ver mis reservas»: es una opción que aparece mientras se revisa una reserva, no un objetivo con el que el cliente entre al sistema.

La decisión que refleja el diagrama es agrupar las operaciones de cada entidad en un solo caso: «Gestionar destinos» reúne registrar, modificar, eliminar y listar. La alternativa descartada era una

elipse por operación, que habría duplicado el diagrama sin agregar información, porque las cuatro operaciones tienen el mismo actor y el mismo objetivo: mantener el catálogo. «Publicar paquete» sí queda aparte, porque no es una edición más: es el momento en que el precio deja de cambiar (R7). La tabla 16 asigna cada requerimiento funcional a su caso de uso; ninguno queda sin caso, y la herramienta de verificación comprueba que la figura y la tabla citen los mismos.

Tabla 16. Casos de uso y requerimientos que cumple cada uno

Caso de uso	Nombre	Actor	Requerimientos que cumple
CU-01	Registrarse	Cliente	RF-RES-01, RF-RES-02, RF-RES-03, RF-SEG-04, RF-SEG-12, RF-SEG-13
CU-02	Iniciar sesión	Cliente y Administrador (incluido)	RF-SEG-01, RF-SEG-02, RF-SEG-03, RF-SEG-05, RF-SEG-08, RF-SEG-09
CU-03	Consultar paquetes	Cliente, con o sin cuenta (S-09)	RF-PAQ-06, RF-PAQ-07
CU-04	Reservar paquete	Cliente	RF-RES-04, RF-RES-05, RF-RES-06, RF-RES-07, RF-RES-10
CU-05	Ver mis reservas	Cliente	RF-RES-08, RF-SEG-10
CU-06	Anular reserva («extend» de CU-05)	Cliente	RF-RES-09
CU-07	Actualizar mis datos	Cliente	RF-RES-12
CU-08	Cambiar contraseña	Cliente y Administrador	RF-SEG-11, RF-SEG-04
CU-09	Gestionar destinos	Administrador	RF-DES-01, RF-DES-02, RF-DES-03, RF-DES-04, RF-DES-05, RF-DES-06, RF-DES-07, RF-DES-08, RF-DES-09, RF-DES-10
CU-10	Gestionar paquetes	Administrador	RF-PAQ-01, RF-PAQ-02, RF-PAQ-03, RF-PAQ-04, RF-PAQ-08, RF-PAQ-09, RF-PAQ-10, RF-PAQ-11
CU-11	Publicar paquete	Administrador	RF-PAQ-05
CU-12	Ver reservas de un paquete	Administrador	RF-RES-11, RF-SEG-10
CU-13	Crear administrador	Administrador (el primero, al instalar)	RF-SEG-06, RF-SEG-07

Las fichas (tabla 17, tabla 18 y tabla 19) detallan lo que no cabe en el dibujo. Se escribieron para los tres casos que concentran las reglas del negocio: el obligatorio, «Reservar paquete», el que fija el precio y el que protege todos los demás. Los flujos alternativos son los que suelen faltar: en ellos aparecen las reglas que en la temporada anterior se violaron.

Tabla 17. Ficha del caso de uso CU-04 Reservar paquete

Caso de uso	CU-04 Reservar paquete
Actor principal	Cliente

Requerimientos	RF-RES-04, RF-RES-05, RF-RES-06, RF-RES-07, RF-RES-10 · RNF-FIA-02, RNF-SEG-05 · reglas R11 a R16
Precondición	El cliente inició sesión (CU-02)
Postcondición	La reserva queda guardada como vigente, con su fecha de emisión y su total fijo; el cupo disponible del paquete baja en la cantidad de personas reservada
Flujo principal	1. El sistema muestra los paquetes disponibles, con su precio por persona y su cupo disponible (CU-03) 2. El cliente elige un paquete e indica la cantidad de personas 3. El sistema calcula el total (precio fijado al publicar por la cantidad de personas) y lo muestra 4. El cliente confirma la reserva 5. El sistema comprueba el cupo y la fecha de salida y guarda la reserva, las dos cosas en una sola transacción 6. El sistema muestra el comprobante: número de reserva, paquete, fechas, personas y total
Flujos alternativos	• 2a. La cantidad no es un número entero o es menor que uno: el sistema lo informa y la vuelve a pedir (R16, RF-RES-05) • 2b. El cliente ya tiene una reserva vigente en ese paquete: el sistema lo advierte y pide confirmación antes de seguir (RF-RES-10) • 5a. La fecha de salida ya llegó: el sistema rechaza la reserva, lo informa y vuelve al listado (R15, RF-RES-06) • 5b. Las personas superan el cupo disponible, por ejemplo porque otra reserva lo tomó entre el paso 1 y el 5: el sistema rechaza la reserva e informa cuántos cupos quedan (R14, RF-RES-05) • En cualquier paso, el cliente escribe x: la operación se cancela sin guardar nada (RNF-USA-01)

Tabla 18. Ficha del caso de uso CU-11 Publicar paquete

Caso de uso	CU-11 Publicar paquete
Actor principal	Administrador
Requerimientos	RF-PAQ-05 · reglas R6 y R7
Precondición	El administrador inició sesión (CU-02) y existe un paquete en borrador (CU-10)
Postcondición	El paquete queda publicado, con su precio por persona fijo: no cambia aunque después cambie el costo de un destino
Flujo principal	1. El sistema muestra los paquetes en borrador 2. El administrador elige uno 3. El sistema muestra sus destinos con el costo base de cada uno, el margen y el precio por persona calculado 4. El administrador confirma la publicación 5. El sistema fija el precio por persona, marca el paquete como publicado y lo informa

Flujos alternativos	3a. Algún destino del paquete quedó no disponible después de armarlo: el sistema no lo publica y lo informa, para que se reemplace el destino (R8, RF-PAQ-02) 4a. El administrador no confirma: el paquete sigue en borrador y su precio sigue calculándose desde los costos vigentes
----------------------------	--

Tabla 19. Ficha del caso de uso CU-02 Iniciar sesión

Caso de uso	CU-02 Iniciar sesión
Actor principal	Cliente o Administrador (incluido por los casos que exigen sesión)
Requerimientos	RF-SEG-01, RF-SEG-02, RF-SEG-03, RF-SEG-05, RF-SEG-08, RF-SEG-09
Precondición	La cuenta existe (CU-01 o CU-13)
Postcondición	La sesión queda abierta con los permisos del rol de la cuenta, y el menú muestra solo las opciones de ese rol
Flujo principal	1. El sistema pide el correo 2. El sistema pide la contraseña sin mostrarla en pantalla 3. El sistema la compara con el resumen guardado 4. El sistema muestra el menú del rol
Flujos alternativos	3a. El correo no existe o la contraseña no coincide: el sistema responde con el mismo mensaje en los dos casos y en el mismo tiempo, sin decir cuál falló, y cuenta el intento fallido (RF-SEG-02) 3b. La cuenta acumula cinco intentos fallidos seguidos: el sistema la bloquea durante cinco minutos y lo informa (RF-SEG-03) 4a. Pasan diez minutos sin actividad, o el usuario pide salir: el sistema cierra la sesión (RF-SEG-08, RF-SEG-09)

Esto implica que cada requerimiento funcional tiene un actor que lo necesita y un objetivo que lo explica, y que las reglas R14 y R15, las de las seis reservas sobre el cupo y las tres sobre fechas vencidas, ya tienen un lugar preciso en el flujo antes de escribir el código.

3.2 Procesos de negocio en BPMN

Responde a 4.1.2.G.5: modela actores, actividades y procesos mediante diagramas BPMN.

El diagrama de clases dice de qué está hecho el sistema; BPMN dice en qué orden ocurre el trabajo y quién ejecuta cada paso. Se usa la notación BPMN 2.0, normada como ISO/IEC 19510: los símbolos significan lo mismo en cualquier organización. Se modelaron los dos procesos que el caso señala como origen de las pérdidas: **reservar un paquete** (seis reservas sobre el cupo y tres sobre fechas vencidas) y **armar y publicar un paquete** (cuatro paquetes cobrados a un precio distinto del publicado). Se agregó un tercero, **anular una reserva**, porque es el vacío que el propio caso nombra (qué ocurre cuando un cliente desiste). Ambos describen el proceso con el sistema funcionando (TO-BE): la figura 2 y la figura 3. La tabla 20 resume los elementos de la notación que se usaron. El proceso actual, con la planilla y el cuaderno, ya está descrito en la sección 1.1 con sus cifras.

Tabla 20. Elementos de BPMN 2.0 usados en los diagramas

Elemento BPMN	Símbolo	Cómo se usó
Pool y carriles	Banda con el nombre del proceso; un carril por participante	Cada tarea está en el carril de quien la ejecuta: Cliente o Administrador, y Sistema. Es lo que distingue un BPMN de un diagrama de flujo
Evento de inicio y de fin	Círculo de borde fino y de borde grueso	Un solo inicio por proceso, nombrado con lo que lo dispara. Cada camino termina en su propio fin
Tarea	Rectángulo redondeado	Verbo y objeto («Calcular el total»). Describe trabajo del negocio, no pasos del código
Compuerta exclusiva	Rombo con una X	Escrita como pregunta, con «sí» y «no» en sus salidas, y la regla que la origina
Flujo de secuencia	Línea continua con punta rellena	Solo líneas horizontales y verticales; ninguna queda suelta ni cruza una tarea

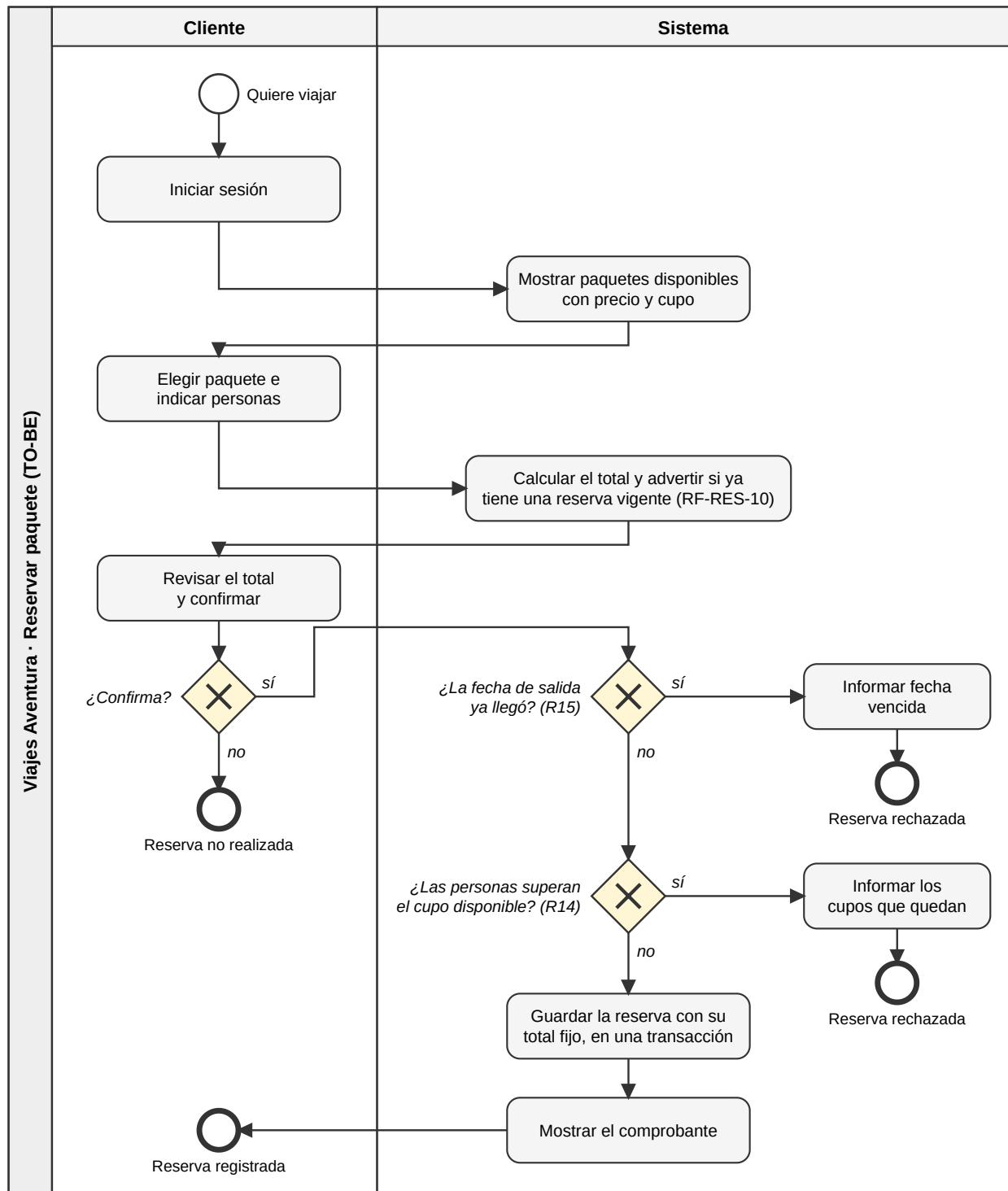


Figura 2. BPMN del proceso «Reservar paquete» (TO-BE): las dos compuertas son las dos fallas de la temporada.

El proceso de reserva alterna entre el cliente, que decide, y el sistema, que muestra, calcula y comprueba. Las dos compuertas están en el carril del sistema, después de la confirmación del cliente, y en ese orden: primero la fecha de salida (R15) y después el cupo (R14). La pista del caso advierte que no son el mismo problema: la fecha se compara contra el día de hoy, y el cupo contra

un dato que el sistema lleva, el cupo máximo menos lo reservado. El cupo se comprueba al confirmar y no solo al mostrar el listado, porque entre las dos cosas otra persona puede haber reservado: por eso la tarea «Guardar la reserva» va en una transacción con la comprobación. Cada rechazo termina en su propio evento de fin, con un mensaje distinto, y ninguno deja la reserva a medias. Antes de confirmar, el cálculo del total advierte si el cliente ya tiene una reserva vigente en ese paquete (RF-RES-10), que es como se duplicaban reservas en el cuaderno (P-01), y el cliente puede no confirmar. Las validaciones de formato (una cantidad que no es un número, por ejemplo) no se modelan: son el flujo 2a de la ficha y no cambian el proceso. La alternativa descartada era comprobar el cupo solo al mostrar la oferta, que deja pasar dos reservas por el mismo último lugar.

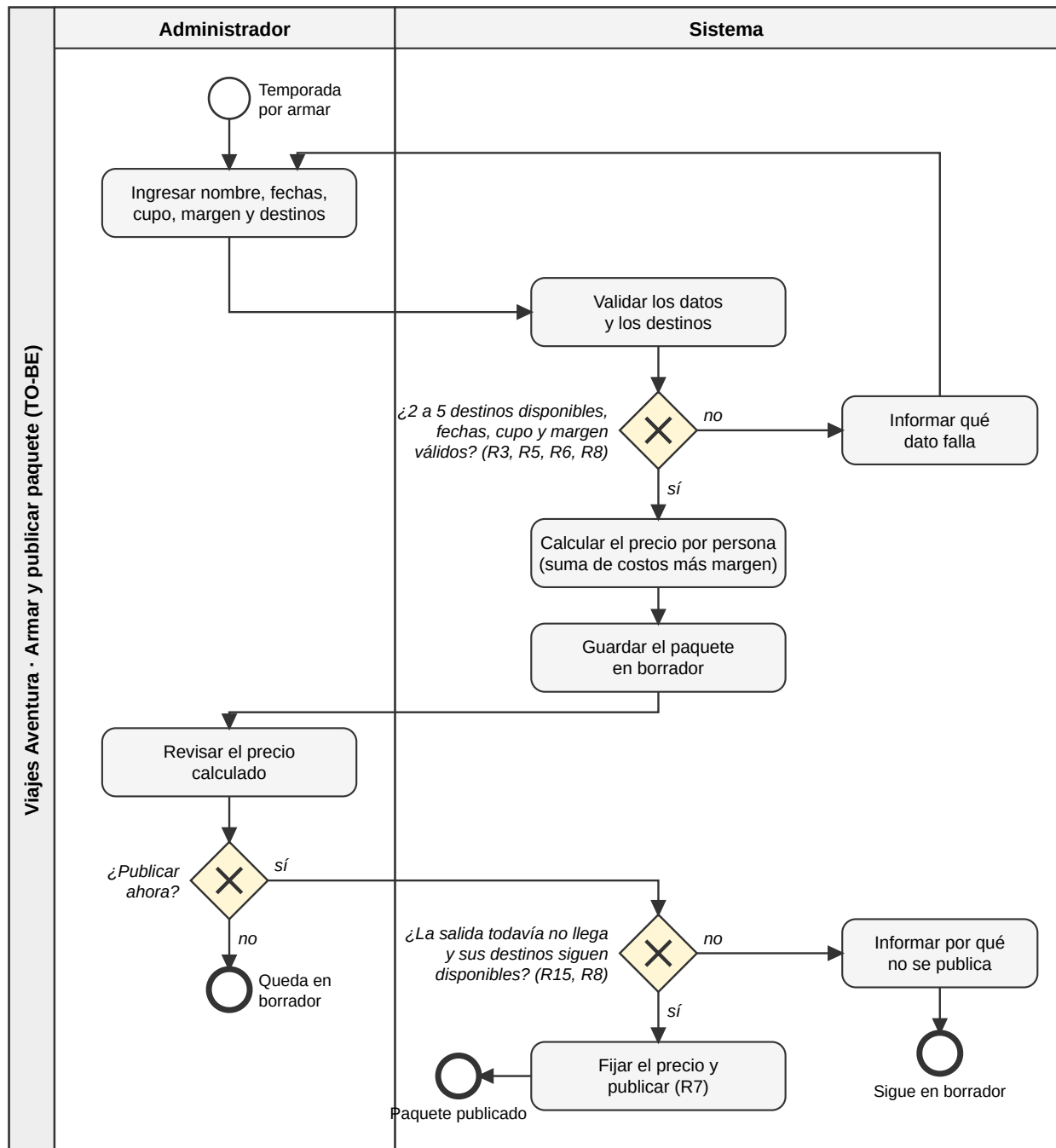


Figura 3. BPMN del proceso «Armar y publicar paquete» (TO-BE): el precio queda fijo recién al publicar.

El proceso del paquete tiene un único ciclo: si la validación falla (menos de dos o más de cinco destinos, un destino no disponible, fechas, cupo o margen inválidos), el sistema dice qué dato falla y el administrador vuelve a ingresarlo. Superada la validación, el sistema calcula el precio y guarda el paquete en borrador; mientras está en borrador, el precio sigue los costos vigentes de sus destinos. La segunda compuerta es la decisión de publicar, y está en el carril del administrador porque es suya: al publicar, el sistema fija el precio por persona y desde ahí ya no cambia (R7). La alternativa descartada era fijar el precio al crear el paquete, que obliga a rehacerlo si un costo cambia antes de

ofrecerlo; o no fijarlo nunca, que es exactamente lo que produjo los cuatro paquetes cobrados a otro precio. Al publicar, el sistema vuelve a comprobar que la salida no haya llegado y que los destinos sigan disponibles (R15 y R8): un destino pudo quedar no disponible mientras el paquete estaba en borrador, y si no se cumple, el paquete sigue en borrador con el motivo informado.

La figura 4 responde qué ocurre cuando un cliente desiste: el vacío que el caso deja abierto y que el supuesto S-01 resuelve.

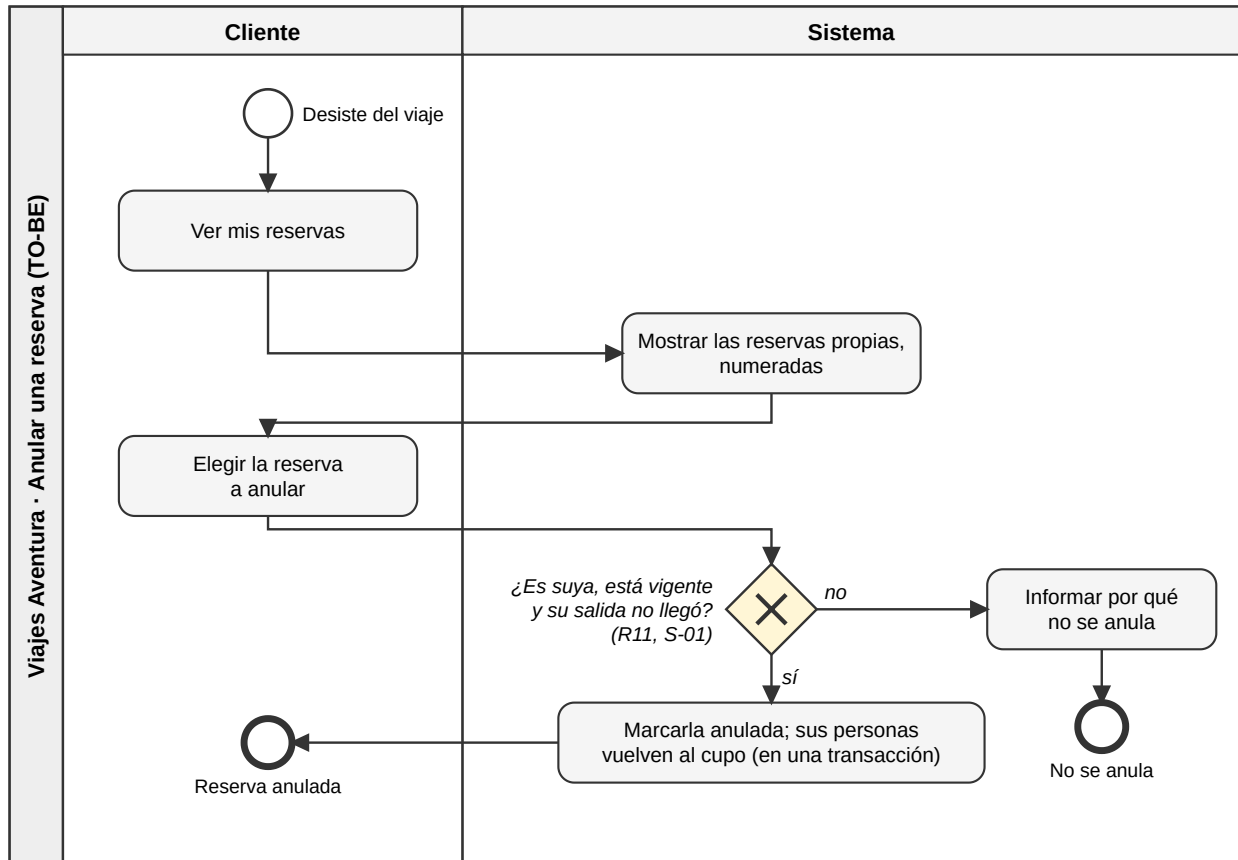


Figura 4. BPMN del proceso «Anular una reserva» (TO-BE): la reserva no se borra, queda anulada.

El cliente elige la reserva desde su propia lista, así que no puede nombrar una ajena. La única compuerta reúne las tres condiciones que el sistema comprueba en la misma sentencia que anula: que la reserva sea suya (R11), que esté vigente y que su salida no haya llegado. Si se cumplen, la reserva queda anulada y sus personas vuelven al cupo; el registro se conserva para el historial (R11) y el total cobrado no cambia (R13). La alternativa descartada era borrar la reserva, que repetiría el problema del cuaderno: no quedaría rastro de lo que el cliente pagó (P-05).

Esto implica que las reglas que costaron dinero en la temporada (R7, R14 y R15), y la anulación que el caso dejó abierta, tienen un punto exacto en el proceso donde se comprueban, que el código del capítulo 5 implementa en ese mismo orden.

3.3 Diagrama de clases

Responde a 4.1.2.G.7: elabora diagramas de clases UML considerando atributos, métodos y relaciones.

El modelo se construyó con el mismo método de la Evaluación Sumativa 1: **una clase existe si y solo si una frase del caso la exige**, y ante dos lecturas igual de válidas gana la que tiene menos elementos. El número de clases no se estimó: salió de recorrer el caso y el catálogo del capítulo 2, incluidas las frases que **no** producen una clase, que son las que explican por qué el modelo no es más grande.

De qué frase sale cada clase

La tabla 21 muestra la frase del caso que exige cada clase.

Tabla 21. Frase del caso que origina cada clase

Clase	Tipo	Frase del caso que la exige	Requerimientos que la usan
Usuario	Abstracta	R11: «Solo un cliente autenticado puede reservar» y §1.3: el administrador también opera el sistema. Los dos inician sesión igual, pero ninguno es «solo un usuario»: por eso es abstracta	RF-SEG-01, RF-SEG-02, RF-SEG-03, RF-SEG-04, RF-SEG-11, RNF-SEG-01
Cliente	Entidad	R9: «Un cliente se registra con nombre, RUT, correo electrónico, teléfono y contraseña»	RF-RES-01, RF-RES-02, RF-RES-03, RF-RES-08, RF-RES-12, RF-SEG-10
Administrador	Entidad	§1.3: «Mantiene el catálogo de destinos, arma los paquetes, fija los cupos y publica las temporadas»	RF-SEG-05, RF-SEG-06, RF-SEG-07
Destino	Entidad	R1: «Un destino tiene nombre, zona, descripción, duración en días y un costo base por persona»	RF-DES-01 a RF-DES-09
Paquete	Entidad	R5: «Un paquete tiene nombre, fecha de salida, fecha de regreso y cupo máximo», y R6 y R7 (margen y precio fijado)	RF-PAQ-01 a RF-PAQ-10
Reserva	Entidad	R12: «Una reserva corresponde a un cliente y a un paquete, y registra la fecha en que se emitió, la cantidad de personas y el total cobrado»	RF-RES-04 a RF-RES-11
EstadoReserva	Enumeración	Supuesto S-01: una reserva anulada no se borra, queda «anulada»	RF-RES-09, RF-PAQ-06
ReglaNegocioError	Excepción	Las reglas del caso que dicen qué «no se acepta» (R3, R5, R6, R14, R15 y R16) y §6: «validación de todo dato ingresado». Cada rechazo lleva el código de la regla que lo causó	RF-DES-02, RF-DES-03, RF-PAQ-02, RF-PAQ-03, RF-RES-02, RF-RES-05, RF-RES-06, RF-SEG-04

Las frases que no producen una clase, y por qué (tabla 22):

Tabla 22. Candidatos descartados como clase

Candidato	Por qué no es una clase
Persona sin cuenta	Puede consultar la oferta (S-09), pero no tiene datos propios ni el sistema la recuerda: no hay nada que guardar de ella

Zona, margen, cupo	El caso no les da datos propios: la zona es un texto del destino (R1), el margen y el cupo máximo son atributos del paquete (R5, R6)
Cupo disponible	Es un dato derivado (R14): se calcula con <code>cupoDisponible()</code> . Guardarlo sería tener dos copias del mismo dato, que es justamente la causa común de la sección 1.1
Rol	Se resuelve con herencia y no con un atributo: ver la decisión 1 más abajo
Pago, pasajero	Fuera del alcance (W-01) y fuera de R12 (S-10)
Menú, sesión, cifrado	Dicen cómo se construye el sistema, no de qué está hecho el negocio. El menú y la sesión son interfaz; el cifrado es una función de apoyo que usa <code>Cliente</code> por dentro

El diagrama

La figura 5 responde de qué piezas está hecho el sistema. Muestra el modelo completo (ocho elementos y 82 miembros) en una sola página de márgenes reducidos, con las clases agrupadas en sus tres módulos. Se dibujó con coordenadas fijas: la letra declarada es la que se imprime (7,5 puntos en los miembros), las líneas son solo horizontales y verticales, y ninguna cruza una caja. Las firmas largas se parten en dos renglones dentro de su caja, en lugar de ensanchar la caja y achicar todo el diagrama. Los miembros y las relaciones se leen de un único archivo fuente, `clases.puml`, el mismo que después se compara contra el código; si el dibujo y la fuente no tienen las mismas relaciones, la generación falla.

Convenciones, las mismas de la Evaluación Sumativa 1: nombres en *camelCase* en el diagrama y en *snake_case* en el código; - privado, # protegido, + público; el método de clase va subrayado y el abstracto en cursiva; el constructor no se dibuja; `void` cuando no devuelve nada. Toda operación que escribe o que expone datos de otros recibe al `solicitante`, y la propia clase comprueba su permiso: así el control de acceso vive en el dominio y no solo en el menú.

En amarillo están las dos clases donde viven las reglas que más costaron a la agencia en la última temporada: `Paquete`, que fija el precio al publicar (R7), y `Reserva`, que fija el total (R13) y controla el cupo (R14). En el módulo «Reservas», el fondo claro marca el núcleo del sistema: es lo único que el cliente viene a hacer.

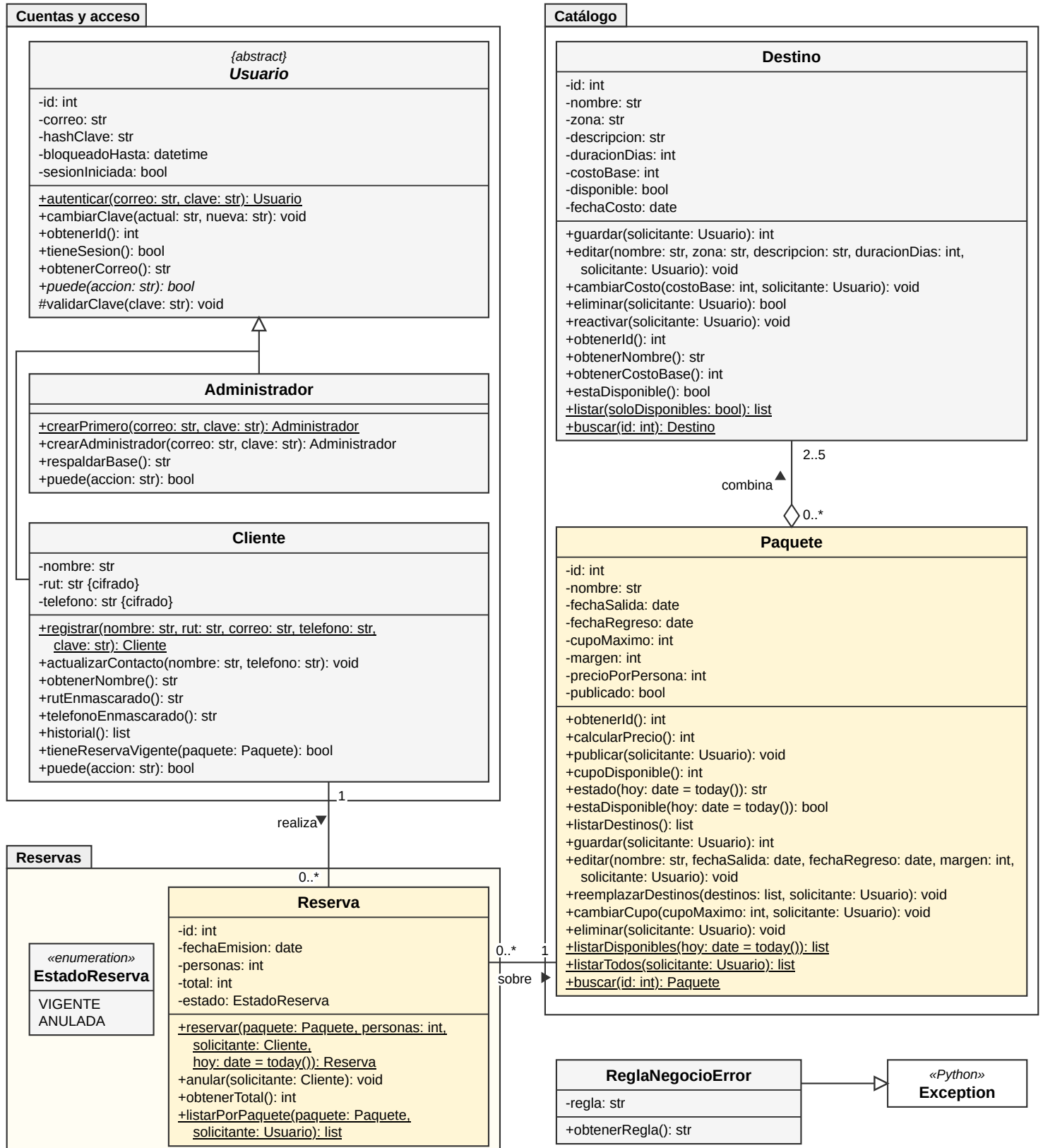


Figura 5. Diagrama de clases completo, en tres módulos.

En la figura se ven la generalización de Usuario en sus dos roles, las tres asociaciones con su multiplicidad y la excepción de las reglas de negocio, que hereda de Exception. Los atributos rut y telefono llevan la restricción {cifrado}: se guardan cifrados (RNF-SEG-02), y rutEnmascarado() y telefonoEnmascarado() son la única forma de mostrarlos (RF-SEG-10). precioPorPersona se fija al publicar (R7), y sesionIniciada la pone en verdadero solo el inicio de sesión (decisión 13).

Relaciones: tipo, multiplicidad y frase de origen

La tabla 23 detalla cada relación con su tipo, su multiplicidad y la frase que la origina.

Tabla 23. Relaciones del diagrama de clases

Relación	Tipo	Multiplicidad	Frase de origen	Por qué ese tipo
Cliente → Usuario	Generalización	(no aplica)	R9 y R11	Un cliente es un usuario que inicia sesión; hereda la autenticación y agrega sus datos personales
Administrador → Usuario	Generalización	(no aplica)	§1.3 y S-03	Un administrador es un usuario con otros permisos. Lo que cambia es el comportamiento de puede(), no los datos
Paquete, «combina», Destino	Agregación	Paquete 0..* · Destino 2..5	R3: «entre dos y cinco destinos» · R4: «Un mismo destino puede formar parte de varios paquetes»	Agregación y no composición porque el destino existe antes que el paquete y lo sobrevive: R8 impide borrarlo si está en un paquete
Cliente, «realiza», Reserva	Asociación	Cliente 1 · Reserva 0..*	R12: «Una reserva corresponde a un cliente»	No es composición: la reserva también pertenece a un paquete, y una parte solo puede tener un dueño. Además se conserva como registro (R11, R13)
Reserva, «sobre», Paquete	Asociación	Reserva 0..* · Paquete 1	R12: «y a un paquete»	Un paquete con reservas no se elimina (RF-PAQ-09): la reserva guarda su total, pero necesita saber qué compró

Decisiones de diseño y alternativas descartadas

La tabla 24 registra cada decisión de diseño con la alternativa que se descartó.

Tabla 24. Decisiones de diseño y alternativas descartadas

#	Decisión	Alternativa descartada	Por qué
1	Los roles son subclases de Usuario, y puede(accion) es abstracto y cada una lo sobrescribe	Un atributo rol y un if por rol en cada método	Con herencia, agregar un rol es agregar una clase, sin tocar las demás; y un cliente no tiene un campo que, alterado, lo convierta en administrador

2	publicado se guarda, pero el estado «vencido» se calcula con estado() desde la fecha	Guardar el estado completo, incluido «vencido»	Publicar es una acción del administrador (R7); vencer no lo es. Si «vencido» dependiera de alguien, se repetiría P-03 (supuesto S-02)
3	precioPorPersona y total se guardan	Calcularlos siempre desde los costos	R7 y R13 exigen que no cambien: calcularlos cada vez reproduciría el caso de Sofía Larraín (P-04)
4	cupoDisponible() se calcula desde las reservas vigentes	Un atributo que se descuenta en cada reserva	Una sola fuente de verdad: un contador aparte podría desincronizarse (P-02)
5	El CRUD va como métodos de cada clase, con el SQL dentro	Un repositorio por clase o un archivo por clase	Es la estructura que el docente pidió en la Unidad 2 (no modularizar clase por clase) y la que se defendió en la Evaluación Sumativa 2
6	Toda operación protegida recibe solicitante y la clase llama a puede() antes de actuar	Controlar el permiso solo en el menú	En la Evaluación Sumativa 2, un permiso que vivía solo en el menú se leyó como «no restringe». Aquí el dominio se protege solo (RF-SEG-05)
7	Todo rechazo de una regla lanza ReglaNegocioError con el código de la regla (R14, R15...), guardado en un atributo privado	Devolver False o lanzar ValueError genérico	El menú distingue el error esperado, que muestra tal cual, del inesperado, que recibe un mensaje genérico sin traza; y cada rechazo queda unido a su regla y a su prueba
8	Los métodos que dependen del día reciben hoy, que por omisión es la fecha del sistema	Llamar a la fecha del sistema dentro del método	La prueba de R15 da el mismo resultado cualquier día que se corra, y el menú no tiene que pasar la fecha
9	margen es un porcentaje entero y el precio se calcula con aritmética entera, redondeado al peso	float, o Decimal con redondeo explícito	El peso no tiene decimales (S-05): el entero no arrastra errores de representación y no necesita una biblioteca más
10	Usuario, Destino y Paquete exponen obtenerId(), de solo lectura	Bajar id a protegido, o que otra clase lea el atributo privado	La reserva necesita el id del cliente y del paquete, y el paquete el de sus destinos. Python oculta los atributos privados entre clases, y un método de lectura mantiene id privado sin permitir cambiarlo. Se agregó al escribir el código (3-oct-2026)

11	El contador de intentos fallidos vive solo en la base; el objeto <code>Usuario</code> guarda <code>bloqueadoHasta</code> , pero no el contador	Un atributo <code>intentosFallidos</code> en el objeto, que el modelo aprobado tenía	Lo suma y lo reinicia una sola sentencia SQL, para que dos sesiones que fallan a la vez no pierdan un intento (S-11). Copiado en el objeto, nadie lo leía (SonarCloud S4487): ante el empate, el modelo con menos elementos. Se quitó el 3-oct-2026
12	<code>Cliente.tieneReservaVigente(paquete)</code> responde si el cliente ya tiene una reserva vigente en ese paquete	Que el menú consulte la base por su cuenta	RF-RES-10 necesita advertir una segunda reserva (P-01), y el menú no contiene SQL: la pregunta le corresponde al cliente, dueño de sus reservas
13	<code>Usuario</code> lleva <code>sesionIniciada</code> , y <code>autorizar()</code> exige una cuenta con sesión además del rol	Revisar solo el rol (<code>puede()</code>)	Con solo el rol, un <code>Administrador</code> armado a mano, sin contraseña, tenía permisos: el criterio de RF-SEG-05 («por cualquier vía») no se cumplía. Lo detectó la auditoría final (3-oct-2026)
14	<code>Administrador.respaldaBase()</code> copia la base con la API de respaldo de <code>sqlite3</code> a una carpeta fija	Copiar el archivo a mano o pedir la ruta al usuario	La API copia una foto coherente aunque otra sesión esté escribiendo, y una carpeta fija evita que un dato del usuario forme una ruta (RNF-FIA-03, P-14)

Principios de la orientación a objetos en el modelo

La tabla 25 muestra dónde se ve cada principio en el diagrama y qué consecuencia tiene para la seguridad.

Tabla 25. Los cuatro principios de la orientación a objetos en el modelo

Principio	Dónde se ve en el diagrama	Consecuencia para la seguridad
Abstracción	<code>Usuario</code> es abstracta: reúne lo común (credenciales, bloqueo, autenticación) y no se puede instanciar	La autenticación y el bloqueo se escriben una sola vez: no hay un rol con un inicio de sesión más débil
Encapsulamiento	Todos los atributos son privados; <code>rut</code> y <code>telefono</code> solo salen enmascarados, <code>hashClave</code> no tiene método de lectura	Ningún código externo lee la credencial ni el RUT en claro, y toda escritura pasa por un método que valida
Herencia	<code>Cliente</code> y <code>Administrador</code> heredan de <code>Usuario</code>	Una corrección en la autenticación protege a los dos roles a la vez
Polimorfismo	<code>puede(accion)</code> tiene una respuesta distinta en cada rol; el menú pregunta sin saber qué rol tiene enfrente	Los permisos no dependen de comparaciones de texto dispersas por el programa, que es donde suelen quedar huecos

Esto implica que el modelo no es una lista de tablas con métodos: cada clase existe por una frase del caso, cada relación tiene su frase y su criterio de ciclo de vida, y las reglas que en la temporada pasada fallaron (cupó, fecha, precio) viven dentro de la clase dueña del dato.

4. Planificación ágil

El proyecto se planificó con Scrum (Schwaber y Sutherland, 2020), adaptado a un equipo de una persona y a un plazo de seis días (del miércoles 30 de septiembre al lunes 5 de octubre de 2026). La adaptación se declara y se justifica en cada pieza, porque Scrum está pensado para equipos de varias personas y aplicarlo sin ajustes sería una formalidad vacía. Como recuerda la guía del curso, Scrum no se aplica «entero o nada»: un equipo pequeño toma la iteración, la lista ordenada y la revisión frecuente, que es donde está el valor.

4.1 Roles y responsabilidades

Responde a 4.1.3.G.9: define roles y responsabilidades dentro del equipo de trabajo.

El trabajo es individual. El **Product Owner** no es quien construye: es el cliente, los socios de Viajes Aventura, cuyas necesidades están en el documento del caso; el docente responde por ellos las consultas. Así quien acepta el producto sigue siendo alguien distinto de quien lo hace. Los demás roles los ocupa una sola persona, y la guía del curso agrega dos responsabilidades técnicas que en un equipo de tres se reparten. El riesgo de juntar roles es el conflicto de intereses: quien construye también decide cuándo algo está terminado. Se controla con dos reglas que no dependen del criterio del momento: los criterios de aceptación (sección 2.5) se escribieron **antes** de programar, y ninguna historia se da por terminada sin su prueba automática en verde (definición de terminado, sección 4.3) (tabla 26).

Tabla 26. Roles y responsabilidades

Rol	Quién	Responsabilidades en este proyecto
Product Owner	Socios de Viajes Aventura (el caso); el docente responde las consultas	Definen el objetivo del producto y el valor de cada requerimiento. Su voz en el proyecto son el caso, sus entrevistas y sus cifras, que ordenan el Product Backlog por MoSCoW (sección 2.4)
Scrum Master y Líder de Proyecto	Leonardo Guerrero	Facilita el método sin dirigirlo: que cada sprint tenga objetivo, que se respete su tiempo y que la revisión y la retrospectiva queden escritas. Como líder de proyecto, coordina el calendario, es el interlocutor con el docente y controla el avance contra el cronograma (4.4)
Developer	Leonardo Guerrero	Modela, programa, prueba y documenta sobre el sistema completo. Usa herramientas de IA como apoyo y valida cada aporte (capítulo de uso de IA)
Responsable de datos	Leonardo Guerrero	Responde por el esquema de las cinco tablas, por la persistencia y por que ninguna consulta se arme concatenando texto
Responsable de seguridad	Leonardo Guerrero	Responde por la autenticación, la validación de entradas, el cifrado de los datos sensibles y por que ninguna credencial quede escrita en el código

La herramienta de IA no es un integrante del equipo: no decide ni acepta nada. Sus aportes entran al código solo después del análisis que registra el capítulo de uso de IA.

4.2 Product Backlog

Responde a 4.1.3.G.10: elabora y prioriza un Product Backlog alineado con los requerimientos identificados.

El backlog está jerarquizado en tres niveles: **épicas** (los módulos del alcance), **historias de usuario** con la forma «como... quiero... para...», y los **requerimientos** del capítulo 2 que cada historia cumple, con sus criterios de aceptación. Una historia hereda la prioridad más alta de sus requerimientos. La estimación usa puntos de historia en la serie de Fibonacci, que miden esfuerzo relativo y no horas (tabla 27).

Objetivo del Producto. Que los socios de Viajes Aventura vendan sus paquetes sin sobreventa, al precio publicado y sin reservas sobre paquetes vencidos, con los datos personales de sus clientes protegidos, en un sistema que reemplace la planilla y el cuaderno desde la temporada siguiente. Cada historia del backlog se ordena por cuánto acerca a ese objetivo: primero las Must, después las Should y al final las Could. HU-00 y HU-23 son historias técnicas: el cliente no las pide, pero habilitan las demás (el repositorio con pruebas automáticas y una prueba por regla), y se dejan en el backlog para que su trabajo quede planificado y a la vista.

Tabla 27. Product Backlog

ID	Épica	Historia de usuario	Requerimientos	Prioridad	Puntos	Sprint
HU-00	Base (técnica)	Como desarrollador quiero un repositorio público con dependencias fijadas y pruebas automáticas, para que cualquiera pueda ejecutar y verificar el sistema	RNF-POR-01	Must	3	S1
HU-01	Seguridad	Como socio quiero crear la primera cuenta de administrador al instalar, para que nadie entre con una clave por defecto	RF-SEG-06, RF-SEG-04	Must	3	S1
HU-02	Reservas	Como cliente quiero registrarme con mis datos, para poder reservar	RF-RES-01, RF-RES-02, RF-RES-03, RF-SEG-12, RNF-LEG-01	Must	5	S1
HU-03	Seguridad	Como usuario quiero iniciar y cerrar sesión con mi correo y mi contraseña, para que solo yo acceda a mi cuenta	RF-SEG-01, RF-SEG-02, RF-SEG-08, RNF-SEG-01, RNF-REN-02	Must	5	S1
HU-04	Seguridad	Como socio quiero que una cuenta se bloquee tras cinco intentos fallidos, para frenar a quien intente adivinar contraseñas	RF-SEG-03	Must	3	S1
HU-05	Seguridad	Como socio quiero que cada rol vea y haga solo lo suyo, para que un cliente no pueda tocar el catálogo	RF-SEG-05, RNF-USA-03	Must	3	S1

HU-06	Seguridad	Como cliente quiero que mi RUT y mi teléfono se guarden cifrados y se muestren enmascarados, para que una copia de la base no me exponga	RF-SEG-10, RF-SEG-13, RNF-SEG-02, RNF-SEG-04	Must	5	S1
HU-10	Destinos	Como administradora quiero registrar destinos sin nombres repetidos ni costos inválidos, para tener un catálogo limpio	RF-DES-01, RF-DES-02, RF-DES-03	Must	3	S1
HU-11	Destinos	Como administradora quiero modificar un destino y que quede la fecha del último cambio de costo, para saber qué tan vigente es	RF-DES-04, RF-DES-05	Must	2	S1
HU-12	Destinos	Como administradora quiero eliminar un destino o dejarlo no disponible si está en un paquete, para no perder lo ya vendido	RF-DES-06, RF-DES-07	Must	3	S1
HU-13	Destinos	Como administradora quiero listar el catálogo con su estado, para ver de un vistazo qué se ofrece	RF-DES-08, RF-DES-10	Must	1	S1
HU-14	Paquetes	Como administradora quiero crear un paquete con dos a cinco destinos y ver su precio calculado, para no sumar a mano	RF-PAQ-01, RF-PAQ-02, RF-PAQ-03, RF-PAQ-04, RF-PAQ-11	Must	5	S2
HU-15	Paquetes	Como administradora quiero publicar un paquete y que su precio quede fijo, para cobrar lo que publiqué	RF-PAQ-05	Must	2	S2
HU-16	Paquetes	Como cliente quiero ver los paquetes disponibles con su cupo, para elegir uno que todavía tenga lugar	RF-PAQ-06, RF-PAQ-07, RNF-REN-01	Must	3	S2
HU-18	Reservas	Como cliente quiero reservar un paquete para varias personas y que no se acepte si no hay cupo o ya salió, para tener mi lugar asegurado	RF-RES-04, RF-RES-05, RF-RES-06, RF-RES-07, RF-RES-10, RNF-FIA-02	Must	8	S2
HU-19	Reservas	Como cliente quiero ver todas mis reservas, para saber qué reservé sin preguntarle a nadie	RF-RES-08	Must	2	S2
HU-23	Calidad (técnica)	Como socio quiero que cada regla del negocio esté protegida por la base y por una prueba automática, para que un error no se repita sin que nadie lo note	RNF-FIA-01, RNF-MAN-01, RNF-MAN-02, RNF-SEG-03	Must	5	S2

HU-24	Calidad	Como usuario quiero poder cancelar cualquier operación y recibir errores claros, sin detalles técnicos, para no dejar nada a medias	RNF-USA-01, RNF-USA-02, RNF-SEG-05	Must	2	S2
HU-07	Seguridad	Como usuario quiero cambiar mi contraseña, para reemplazarla si sospecho que se conoce	RF-SEG-11	Should	2	S1
HU-08	Seguridad	Como socio administrador quiero crear cuentas de administrador para mis socios, para que cada uno entre con la suya	RF-SEG-07	Should	2	S2
HU-09	Seguridad	Como socio quiero que la sesión se cierre sola tras diez minutos sin uso, para que nadie use una sesión abierta en el equipo de la oficina	RF-SEG-09	Should	1	S1
HU-17	Paquetes	Como administradora quiero modificar, eliminar y listar paquetes, para mantener la oferta al día	RF-PAQ-08, RF-PAQ-09, RF-PAQ-10	Should	3	S2
HU-20	Reservas	Como cliente quiero anular una reserva antes de la salida, para liberar el cupo si desisto	RF-RES-09	Should	2	S2
HU-21	Reservas	Como socio quiero ver las reservas de un paquete, para saber quiénes viajan	RF-RES-11	Should	2	S2
HU-26	Calidad	Como socio quiero respaldar la base de datos desde el menú, para no perder las reservas si falla el equipo de la oficina	RNF-FIA-03	Should	2	S3
HU-22	Reservas	Como cliente quiero actualizar mi nombre y mi teléfono, para que la agencia pueda contactarme	RF-RES-12	Could	1	S2
HU-25	Destinos	Como administradora quiero volver a ofrecer un destino que estaba no disponible, para reutilizarlo en paquetes nuevos	RF-DES-09	Could	1	S2

El trabajo que no es una funcionalidad para un usuario (modelar, auditar, escribir el informe, preparar la defensa) no se disfraza de historia: va como **tarea técnica** (TT) en el sprint que corresponde.

4.3 Sprint Backlog

Responde a 4.1.3.G.11: organiza iteraciones de trabajo mediante Sprint Backlog.

Tres sprints de un día y un cierre. Cada sprint termina con un **incremento que se ejecuta desde el menú**: primero las cuentas y los destinos, después los paquetes y las reservas, y al final el sistema auditado y endurecido. No hay un sprint «de análisis» ni uno «de programación»: en cada uno se modela, se construye y se prueba la parte que entrega. Las historias Should y Could van al final de

cada sprint: si el tiempo no alcanza, se sacrifican primero, sin tocar ninguna Must. Tres de ellas (RF-PAQ-08, RF-PAQ-09 y RF-RES-12) se comprometieron igual en su sprint, porque completan el CRUD de su entidad (G.15); su prioridad no cambia, porque dice cuánto le importa al cliente, no en qué orden se construye.

Antes del primer sprint hubo un período de **preparación** (del 30 de septiembre al 2 de octubre), que no se cuenta como sprint porque no entregó nada ejecutable: el refinamiento del Product Backlog que la Guía Scrum describe como actividad continua, aquí concentrado al inicio. Produjo el catálogo de requerimientos, el modelo de clases y este backlog, y se declara con sus horas reales (tabla 28).

Tabla 28. Sprint Backlog y período de preparación

Sprint	Fechas	Objetivo	Historias y tareas	Puntos	Horas est.	Entregable
Prep.	30-sep a 2-oct	Refinamiento del Product Backlog: requerimientos, modelo de clases y backlog listos para el primer sprint	TT-01 requerimientos (cap. 1) · TT-04 diagrama de clases · TT-05 planificación ágil	0	4	Capítulos 1 a 4 del informe
S1	Plan: 2-oct, 19:00 a 24:00 Real: 3-oct, 17:15 a 19:26	Incremento 1: un socio instala el sistema, crea su cuenta, los clientes se registran con sus datos cifrados, y el catálogo de destinos se mantiene desde el menú	TT-02 casos de uso y fichas · TT-03 BPMN · HU-00 · HU-01 · HU-02 · HU-03 · HU-04 · HU-05 · HU-06 · HU-10 · HU-11 · HU-12 · HU-13 · HU-07 · HU-09	39	6	Repositorio con CI; viajes.py y main.py con cuentas y destinos, autoverificación en verde
S2	Plan: 3-oct Real: 3-oct, 19:26 a 19:57	Incremento 2: un cliente reserva un paquete publicado sin exceder el cupo, con todas las reglas R1 a R17 protegidas por prueba	HU-14 · HU-15 · HU-16 · HU-18 · HU-19 · HU-23 · HU-24 · HU-17 · HU-20 · HU-21 · HU-08 · HU-22 · HU-25	38	10	Aplicación completa; prueba_rubrica.py y el workflow en verde
S3	Plan: 4-oct Real: 3-oct, 19:57 a 22:40	Incremento 3: el sistema auditado con IA y endurecido, con cada hallazgo corregido o declarado, y documentado	TT-06 auditoría de seguridad · TT-07 registro de IA · TT-08 informe · TT-09 guion y preguntas de la defensa · HU-26	2	10	AUDITORIA.md, ANALISIS_IA.md, informe en PDF, material de defensa
Cierre	4 y 5-oct	Entregar y defender	TT-10 limpieza, ENTREGA.md y subida al AAI antes de las 23:00 · defensa individual	0	3	Entrega en el AAI

Definición de terminado. Una historia está terminada cuando: su código implementa el diagrama de clases sin diferencias; cada criterio de aceptación de sus requerimientos tiene una prueba en verde; el workflow de GitHub Actions pasa; el informe y el registro de IA están al día; y está integrada

en main desde su rama de trabajo, con commits que citan su identificador (por ejemplo, feat(reservas): HU-18 rechazar reserva sobre el cupo (R14)). El historial de commits es así la evidencia del avance de cada sprint. En un equipo de tres, la definición de la guía del curso pide además que otro integrante pruebe la funcionalidad; aquí ese papel lo cumple la prueba automática, que no conoce las intenciones de quien escribió el código.

La tabla 29 muestra cómo se adapta cada evento de Scrum a un equipo de una persona.

Tabla 29. Eventos de Scrum adaptados a un equipo de una persona

Evento de Scrum	Cómo se adapta a un equipo de una persona
Planificación del sprint	Al inicio de cada sprint se revisa su fila en esta tabla y se confirma el objetivo
Reunión diaria	Con sprints de un día no aplica como reunión; su función (detectar bloqueos) la cumple el registro de trabajo, que anota cada instrucción con su hora y se lleva aparte del repositorio
Revisión del sprint	Se ejecutan las pruebas de las historias del sprint contra sus criterios de aceptación
Retrospectiva	Tres líneas escritas al cierre, en este informe

Tablero al cierre de cada sprint. El avance se lleva en un tablero de cuatro columnas: por hacer, en curso, en revisión (integrada en main, a la espera del workflow en verde) y terminado, entendiendo por terminado lo que cumple la definición anterior. La tabla 30 es una foto del tablero al cerrar cada período: muestra lo que ocurrió, no lo planeado.

Tabla 30. Tablero al cierre de cada período

Cierre	Por hacer	En curso	En revisión	Terminado
Prep.	Todas las HU; TT-02, TT-03 y TT-06 a TT-10	Ninguna	Ninguna	TT-01 requerimientos · TT-04 diagrama de clases (aprobado el 2-oct) · TT-05 planificación ágil
S1	HU-14 a HU-21, HU-23 a HU-25; TT-06 a TT-10	HU-06 (falta la prueba de los permisos 0600) · HU-09 (falta la prueba de la inactividad)	Ninguna	TT-02 casos de uso · TT-03 BPMN · HU-00 · HU-01 · HU-02 · HU-03 · HU-04 · HU-05 · HU-07 · HU-10 · HU-11 · HU-12 · HU-13 · adelantadas del S2: HU-08 y HU-22
S2	TT-06 a TT-10	Ninguna	Ninguna	HU-06 y HU-09 (del S1) · HU-14 a HU-21 · HU-23 · HU-24 · HU-25 · más HU-08 y HU-22, adelantadas en el S1
S3	TT-10 subida al AAI y defensa (cierre)	Ninguna	Ninguna	TT-06 auditoría de seguridad (de la IA y final) · TT-07 registro de IA · TT-08 informe · TT-09 guion y preguntas de la defensa · HU-26 respaldo · adelantados del cierre: README.md y ENTREGA.md

La tabla 31 registra, al cierre de cada sprint, las horas reales contra las estimadas, la revisión de las historias contra la Definición de terminado y la retrospectiva.

Tabla 31. Revisión y retrospectiva de cada sprint

Sprint	Horas est.	Horas reales	Revisión	Retrospectiva
S1	6	2,2	13 de 15 historias terminadas, más 2 adelantadas del S2. HU-06 y HU-09 tienen el código pero no la prueba que exige la Definición de terminado: pasan al S2. Autoverificación con 4 mutaciones detectadas, sesión del menú con los dos roles y workflow en 6 de 6 combinaciones (Windows, macOS y Linux con Python 3.12 y 3.14)	Funcionó: escribir la prueba junto con cada historia; tres errores los encontraron las propias pruebas antes del commit. No funcionó: el sprint empezó un día tarde, porque la preparación (casos de uso y BPMN) se extendió al 2-oct. Se cambia: el S2 parte con las dos pruebas pendientes del S1, antes de las historias nuevas.
S2	10	0,5	Las 13 historias del sprint y las 2 que venían del S1, terminadas. Aplicación completa desde el menú con los dos roles; diagrama y código sin diferencias (8 clases, 79 miembros); prueba por indicador con 31 afirmaciones; 6 de 6 combinaciones del workflow en verde; SonarCloud sin observaciones abiertas. Dos errores reales encontrados y corregidos: el historial armaba la reserva con un id equivocado, y en una instalación nueva el primer cliente no podía registrarse	Funcionó: probar cada historia por el mismo camino que usa el menú; los dos errores reales los encontraron el recorrido del menú y la prueba por indicador, no la autoverificación. No funcionó: las primeras pruebas preparaban la clave de cifrado de antemano y escondían el orden real de una instalación nueva. Se cambia: toda prueba parte como el sistema real, sin nada preparado por fuera, y el S3 abre con la auditoría de seguridad.
S3	10	2,7	Las cuatro tareas del sprint y HU-26, terminadas, más dos del cierre adelantadas (README . md y ENTREGA . md). Dos auditorías: la de la IA, con 17 hallazgos (16 corregidos, 1 descartado), y la final, con 29 hallazgos y las 30 observaciones de un corrector independiente (24 tomadas, 6 descartadas con su motivo). 27 de 27 mutaciones detectadas; matriz de trazabilidad con los 63 requerimientos; informe sin anotaciones abiertas; workflow en verde	Funcionó: comprobar cada observación de la IA con una prueba o un comando antes de aceptarla; así se descartaron 7 de 47, y se encontró que una regla (el permiso sin sesión iniciada) no la vigilaba ninguna prueba. No funcionó: el sprint estaba planificado para el 4-oct y se hizo el 3-oct, a continuación del S2; el plan por días no siguió el ritmo real. Se cambia: el cierre se usa para ensayar la defensa con el guion y para entregar con margen.

La tabla 32 divide cada sprint en sus tareas, con la hora real de inicio y fin que deja el historial de commits del 3 de octubre (el registro de trabajo da las horas previas al primer commit). Las horas reales de los tres sprints suman 5,4 contra 26 estimadas: la estimación supuso escribir el código a mano, y el asistente de IA lo generó; el tiempo se fue a decidir y a verificar (capítulo 6). Con 79 puntos terminados en 5,4 horas, la próxima estimación se hace con esa velocidad y no en horas de escritura.

Tabla 32. Tareas de cada sprint con su hora real

Sprint	Tarea	Historias	Hora real (3-oct)	Minutos
S1	TT-02 y TT-03: casos de uso, fichas y BPMN; estructura del repositorio	HU-00	17:15 a 18:24	69

S1	Entorno con dependencias fijadas, workflow y sellos	HU-00	18:24 a 18:29	5
S1	Catálogo de destinos y esquema de la base	HU-10 a HU-13	18:29 a 19:12	43
S1	Cuentas, menú por rol y comparador del diagrama con el código	HU-01 a HU-07, HU-23	19:12 a 19:26	14
S2	Pruebas pendientes del S1 (permisos 0600 e inactividad)	HU-06, HU-09	19:26 a 19:29	3
S2	Paquetes, reservas y su menú	HU-14 a HU-21	19:29 a 19:39	10
S2	Prueba por indicador y corrección del primer cliente	HU-23, HU-06	19:39 a 19:57	18
S3	TT-06: auditoría de seguridad de la IA y sus correcciones	HU-23, HU-24	19:57 a 20:29	32
S3	TT-07: análisis del uso de la IA		20:29 a 20:50	21
S3	Adelantado del cierre: README . md y ENTREGA . md	HU-00	20:50 a 20:58	8
S3	TT-08 capítulos 5 a 8; TT-06 auditoría final con corrector independiente y sus correcciones	HU-23, HU-24, HU-26	20:58 a 22:14	76
S3	TT-08 cierre del informe y TT-09 guion y preguntas de la defensa		22:14 a 22:40	26

4.4 Cronograma y entregables

Responde a 4.1.3.1.12: define tiempos y entregables para el desarrollo de la solución.

El plazo lo fija la entrega del lunes 5 de octubre a las 23:00, con la defensa el mismo día. El cronograma trabaja hacia atrás desde esa fecha y deja la meta interna a las 20:00, tres horas antes del cierre del AAI (tabla 33).

Tabla 33. Cronograma por día

Trabajo	Mié 30	Jue 1	Vie 2	Sáb 3	Dom 4	Lun 5	Entregable
Preparación · refinamiento del backlog							Capítulos 1 a 4 (requerimientos, modelos, backlog)
S1 · Cuentas y destinos							Incremento 1 ejecutable, repositorio con CI (planificado el viernes, hecho el sábado)
S2 · Paquetes y reservas							Incremento 2: aplicación completa y probada
S3 · Auditoría, informe y defensa							Incremento 3: sistema auditado; informe y guion (planificado el domingo, hecho el sábado)
Cierre · entrega y defensa							Ensayo de la defensa (domingo); AAI (meta 20:00, cierre 23:00) y defensa (lunes)

El plazo es corto frente a los sprints de una a cuatro semanas que describe la guía del curso. Se sostiene por dos razones medibles: el alcance está acotado por el caso (cuatro entidades, dos roles, terminal y base local) y una parte del código se reutiliza de la Evaluación Sumativa 2, ya auditada (conexión a la base, permisos y manejo de errores). El jueves 1 no hubo trabajo en el proyecto, y así queda en el cronograma.

Esto implica que cada día termina con algo verificable y que el orden no es arbitrario: sigue las dependencias del capítulo 2 (primero las cuentas, después el catálogo, al final las reservas). Si un sprint se atrasa, el recorte ya está decidido: caen primero las historias Could y Should del sprint, nunca una Must.

5. Implementación

Este capítulo muestra cómo el modelo del capítulo 3 se convirtió en una aplicación que funciona, y cómo se comprueba. El código está en el repositorio público github.com/leonardoguerrero-dev/P00-Viajes-Aventura; aquí van solo los fragmentos que ilustran una decisión. Cada afirmación remite a la prueba que la verifica, y todas esas pruebas corren en cada envío al repositorio.

5.1 Estructura del código y cómo se ejecuta

La tabla 34 resume los archivos. El dominio y el menú van en dos archivos, no en uno por clase: es la estructura que pidió el docente en la Unidad 2, y la que se defendió en la Evaluación Sumativa 2.

Tabla 34. Archivos del repositorio

Archivo	Líneas	Qué contiene
<code>viajes.py</code>	1.650	Validaciones compartidas, esquema y conexión a la base, las ocho clases del diagrama con su persistencia y la autoverificación de las reglas R1 a R17. Ninguna línea de SQL vive fuera de este archivo
<code>main.py</code>	645	El menú de terminal: inicio, registro, sesión y un menú por rol. No contiene SQL
<code>herramientas/driver.py</code>	233	Recorre el menú real con un guion de respuestas y guarda la sesión en <code>docs/SALIDA_TERMINAL.md</code>
<code>herramientas/uml_vs_codigo.py</code>	170	Compara el diagrama (<code>diagramas/clases.puml</code>) con el código
<code>herramientas/mutaciones.py</code>	133	Rompe cada regla principal por vez y exige que alguna prueba lo detecte
<code>pruebas/prueba_rubrica.py</code>	318	Una afirmación verificable por indicador de la rúbrica
<code>requirements.txt</code> y <code>.github/workflows/pruebas.yml</code>		Dependencias fijadas con el hash de cada archivo publicado, y el workflow que corre todo lo anterior en Windows, macOS y Linux con Python 3.12 y 3.14

Para ejecutarlo, la sección 1 del `README.md` da los comandos de Windows, macOS y Linux: crear un entorno virtual, instalar las dependencias con `pip install --only-binary :all: --require-hashes -r requirements.txt` y ejecutar `python main.py`. Si falta el entorno virtual o la versión de

Python es anterior a la 3.12, el programa lo dice en una línea en vez de mostrar una traza. La primera vez la base está vacía y el sistema pide crear la cuenta del primer administrador (S-04); no hay ninguna contraseña escrita en el código.

Esto implica que el corrector puede reproducir todo en su propio sistema operativo siguiendo una sola sección del README, y que cada una de las tres capas del sistema (menú, dominio y base) vive en un solo archivo.

5.2 Del diagrama al código

Responde a 4.1.4.G.13: implementa la solución respetando el modelo UML y los principios de programación orientada a objetos.

La fidelidad al diagrama no se afirma: se mide. `herramientas/uml_vs_codigo.py` lee `clases.puml` y el código con el analizador de Python (`ast`, sin ejecutarlo) y compara clase por clase la herencia, cada atributo privado, cada método con sus parámetros en orden y las marcas de método de clase y abstracto. También falla si el código tiene un método público o un atributo público que el diagrama no dibuja. El resultado es **8 clases y 79 miembros comparados, 0 diferencias**, y el workflow lo exige en cada envío. Se probó con 8 mutaciones (un parámetro renombrado, un método o atributo de más, un `@abstractmethod` borrado, una herencia rota, un valor de la enumeración que falta) y las 8 se detectaron.

Las convenciones son las de la Evaluación Sumativa 1: *camelCase* en el diagrama y *snake_case* en el código, que el comparador traduce; `#` protegido equivale a un guion bajo; el constructor, los métodos especiales de Python (`__str__`, `__repr__`) y los auxiliares privados de persistencia no se dibujan. La tabla 35 muestra dónde está cada principio en el código.

Tabla 35. Los cuatro principios de la orientación a objetos en el código

Principio	En el código	Cómo se comprueba
Abstracción	<code>class Usuario(ABC)</code> con <code>puede()</code> marcado <code>@abstractmethod</code> : no existe «solo un usuario»	Instanciar <code>Usuario</code> lanza <code>TypeError</code> (prueba por indicador, G.13)
Encapsulamiento	Todos los atributos son privados (<code>self.__rut</code>); el RUT y el teléfono solo salen enmascarados, y la contraseña ni siquiera tiene método de lectura	El comparador falla ante un atributo público; la prueba revisa que <code>rut</code> , <code>hash_clave</code> y otros no sean accesibles
Herencia	<code>Cliente</code> y <code>Administrador</code> heredan la autenticación, el bloqueo y el cambio de contraseña de <code>Usuario</code> ; <code>ReglaNegocioError</code> hereda de <code>Exception</code>	El comparador revisa cada herencia del diagrama
Polimorfismo	<code>puede(accion)</code> responde distinto en cada rol. El menú se arma preguntándole a la sesión, sin saber qué rol tiene enfrente	La misma llamada da resultados distintos para un cliente y un administrador (G.13); la sesión del menú muestra los dos menús

```
def opciones_de(sesion: Usuario) -> list[tuple]:
    """Solo lo que este rol puede hacer. El dominio vuelve a revisarlo al escribir."""
    return [o for o in OPCIONES if o[2] is None or sesion.puede(o[2])]
```

Después de su aprobación, el modelo cambió ocho veces, y cada cambio quedó como decisión en la sección 3.3 con su motivo (decisiones 7 a 14): `ReglaNegocioError`, la fecha hoy como parámetro, el margen entero, `obtenerId()`, el contador de intentos solo en la base, `tieneReservaVigente()`, la sesión iniciada como condición de todo permiso y el respaldo de la base. Además, `Destino.obtenerNombre()` se agregó para mostrar los destinos de un paquete.

Esto implica que el diagrama de la sección 3.3 no es un dibujo previo que el código fue dejando atrás: es el mismo contrato, verificado por un programa, y cualquier diferencia futura detiene el workflow.

5.3 Persistencia

Responde a 4.1.4.G.14: integra mecanismos de persistencia para gestionar destinos, paquetes y reservas.

La base es un archivo `sqlite3` (biblioteca estándar, sin servidor), con seis tablas: las cinco del dominio y la de auditoría que agregó la evaluación de seguridad (sección 5.9). Las reglas que se pueden expresar en la base se repiten ahí, para que un error del código no deje pasar un dato inválido (RNF-FIA-01): 17 restricciones `CHECK`, el `UNIQUE` del correo (R9) y del nombre normalizado del destino (R1), una clave primaria compuesta que impide repetir un destino en un paquete (R3), 5 claves foráneas activadas en cada conexión e índices en las claves que consultan el historial y el cupo. La tabla 36 resume el esquema.

Tabla 36. Esquema de la base y las reglas que protege

Tabla	Restricciones principales	Reglas
usuario	Correo único y en minúsculas; el hash debe empezar con <code>\$argon2id\$</code> ; un cliente tiene nombre, RUT y teléfono, y un administrador no tiene ninguno	R9, R10, R17
destino	Nombre normalizado único; duración entre 1 y 365 días; costo entre 1 y 100.000.000; disponible 0 o 1	R1, R2, R8
paquete	Regreso posterior a la salida; cupo entre 1 y 1.000; margen entre 0 y 1.000 %; si está publicado, tiene precio	R5, R6, R7
paquete_destino	Clave primaria (paquete, destino); el destino no se borra en cascada	R3, R4, R8
reserva	Personas entre 1 y 1.000; total mayor que cero; estado VIGENTE o ANULADA; el paquete no se borra en cascada	R12, R13, R16, S-01
auditoria	Fecha en UTC, usuario, acción y detalle sin datos personales	H-02 (sección 5.9)

Transacciones. Cada operación abre su conexión con `BEGIN IMMEDIATE`, que toma el permiso de escritura antes de la primera consulta. Así, «revisar y después escribir» es una sola unidad: entre que se lee el cupo y se guarda la reserva, nadie más puede reservar. Por omisión, el módulo `sqlite3` abre la transacción recién en la primera escritura, y la consulta previa quedaba afuera. Lo detectó la auditoría con IA (hallazgo H-08) y se comprobó por experimento antes de corregirlo.

```
with conectar() as con:
    # BEGIN IMMEDIATE
    fila = con.execute("SELECT ... cupo_maximo - (personas vigentes) ...").fetchone()
    if personas > fila["disponible"]:
        raise ReglaNegocioError("R14", f"No hay cupo: quedan {fila['disponible']} lugares")
    con.execute("INSERT INTO reserva ...") # la misma transacción: COMMIT al salir
```

La autoverificación lo prueba con dos conexiones que compiten por el último lugar: la segunda espera, ve el cupo ya descontado y se rechaza (RNF-FIA-02). **Eficiencia:** con diez veces el volumen de una temporada (2.140 reservas, 180 destinos y 120 paquetes), la oferta se lista en 0,14 segundos y el historial de un cliente en 0,02 (RNF-REN-01 pide menos de 1).

Escrituras confirmadas y respaldo. Una escritura que actualiza un registro confirma que tocó exactamente una fila: si otra sesión lo eliminó entre medio, el sistema lo informa en vez de decir «guardado». El administrador puede respaldar la base desde el menú (`respaldarBase()`, RNF-FIA-03): la API de respaldo de `sqlite3` copia una foto coherente aunque otra sesión esté escribiendo, en una carpeta fija junto a la base y con permisos solo para su dueño. La clave de cifrado no va en esa copia, para que un respaldo no lleve dato y clave juntos.

Esto implica que las tres fallas que más dinero le costaron a la agencia (la sobreventa, el precio que cambia y la reserva sobre un paquete vencido) están cerradas dos veces: en el código, y en la base, que las rechaza aunque alguien escriba en ella directamente.

5.4 Operaciones CRUD

Responde a 4.1.4.G.15: desarrolla funcionalidades CRUD operativas para la gestión de información.

La tabla 37 cruza cada entidad con las cuatro operaciones. Cada celda nombra el método y está ejercida por la prueba por indicador (G.15) y por la sesión real del menú (`docs/SALIDA_TERMINAL.md`), que recorre el sistema con los dos roles sin un solo error inesperado.

Tabla 37. Matriz de operaciones CRUD por entidad

Entidad	Crear	Leer	Actualizar	Eliminar
Destino	<code>guardar()</code>	<code>listar()</code> , <code>buscar()</code>	<code>editar()</code> , <code>cambiarCosto()</code> , <code>reactivar()</code>	<code>eliminar()</code> : borra, o deja no disponible si está en un paquete (R8)
Paquete	<code>guardar()</code>	<code>listarDisponibles()</code> , <code>listarTodos()</code> , <code>buscar()</code>	<code>editar()</code> , <code>reemplazarDestinos()</code> , <code>publicar()</code> , <code>cambiarCupo()</code>	<code>eliminar()</code> , solo sin reservas (RF-PAQ-09)
Cliente	<code>registrar()</code>	<code>autenticar()</code> , <code>historial()</code>	<code>actualizarContacto()</code> , <code>cambiarClave()</code>	No se elimina: la supresión se atiende a mano (riesgo declarado en la auditoría)
Reserva	<code>reservar()</code>	<code>historial()</code> , <code>listarPorPaquete()</code>	<code>anular()</code>	No se borra: queda anulada, como historial (S-01)

Esto implica que las cuatro operaciones existen donde el negocio las necesita, y donde una no existe (borrar una reserva o un cliente) es por una decisión escrita, no por un olvido.

5.5 Validación en tres capas

Responde a 4.1.4.G.13 y 4.1.4.G.15.

Cada dato se valida tres veces, y cada capa tiene un propósito distinto (tabla 38). La segunda es la que importa para la seguridad: un objeto inválido no llega a existir, se cree desde el menú, desde una prueba o desde la base.

Tabla 38. Las tres capas de validación

Capa	Qué revisa	Ejemplo
Menú (main.py)	Formato, apenas se escribe el dato, para que el usuario corrija en el momento	El RUT con el dígito verificador malo se rechaza y se vuelve a pedir; un número de 5.000 dígitos no llega a <code>int()</code>
Dominio (constructor y métodos)	Tipo y rango, sin depender del menú. Un rechazo de regla lanza <code>ReglaNegocioError</code> con su código	<code>entero()</code> rechaza <code>True</code> como duración; el cupo fuera de rango cita R5
Base (CHECK, UNIQUE, claves foráneas)	La última barrera, aunque se escriba por SQL directo	La prueba inserta seis datos inválidos por SQL y la base rechaza los seis (G.14)

Esto implica que un error en una capa no se convierte en un dato malo guardado: lo detiene la siguiente.

5.6 Acceso seguro: autenticación

Responde a 4.1.5.G.17: implementa mecanismos seguros de autenticación utilizando librerías oficiales.

Las contraseñas se guardan con **Argon2id** mediante `argon2-cffi` 25.1.0, del repositorio oficial PyPI, la primera opción que recomienda OWASP. Cada una lleva su propia sal y un costo de 64 MiB de memoria y 4 pasadas: verificar una toma unos 120 ms, dentro de lo que pide RNF-REN-02 y caro para quien intente adivinar. El hash guarda sus parámetros: si mañana se suben, la cuenta se rehace sola al entrar (`check_needs_rehash`). La contraseña se escribe sin eco (`getpass`) y la sesión se cierra a pedido o tras 10 minutos sin actividad, también si la espera ocurre en medio de una acción (RF-SEG-08 y RF-SEG-09). La primera cuenta se crea en el primer uso, con una sola sentencia que solo inserta si la tabla está vacía: no hay credenciales por omisión (S-04). Todo permiso exige, además del rol, una cuenta con la sesión iniciada: un objeto `Administrador` armado a mano, sin pasar por la contraseña, no puede tocar nada (decisión 13).

Esto implica que una copia de la base no entrega ninguna contraseña, y que los parámetros de seguridad pueden subir sin pedirle a nadie que cambie la suya.

5.7 Validación de credenciales

Responde a 4.1.5.G.18: valida rigurosamente las credenciales de acceso al sistema.

La tabla 39 recorre cada escenario de inicio de sesión con la respuesta del sistema y la prueba que lo verifica.

Tabla 39. Escenarios de autenticación

Escenario	Respuesta del sistema	Prueba
Correo y contraseña correctos	Entra, sin importar mayúsculas en el correo; reinicia el contador	G.18

Contraseña errónea, correo inexistente, correo mal escrito o vacío	El mismo mensaje en los cuatro casos. El correo inexistente verifica un hash señuelo, así que tarda lo mismo (127 ms contra 120 ms medidos): no se puede deducir qué correos existen	G.18, sesión del menú
Cinco fallos seguidos	Bloqueo de 5 minutos guardado en la base, en hora UTC, aun con la contraseña correcta; el contador sube en una sola sentencia, así que dos sesiones a la vez no se saltan intentos	G.18, autoverificación
Contraseña nueva débil	Se rechaza con menos de 12 caracteres, igual al correo, común (lista local) o con menos de 5 caracteres distintos	G.18, autoverificación
Cambio de contraseña	Exige la actual y que la nueva sea distinta; una sesión vieja no puede cambiar una contraseña que otra sesión ya cambió	G.18, autoverificación
Hash con parámetros antiguos	Se rehace al entrar	G.18
Permiso	Un cliente que llama directo a un método del administrador, o una cuenta sin sesión iniciada, recibe <code>PermissionError</code> : el permiso se revisa en el dominio, no solo en el menú	G.13, autoverificación

Esto implica que cada escenario de autenticación que la rúbrica pide tiene una respuesta definida y una prueba que la sostiene, y que ninguno revela información a quien está probando contraseñas.

5.8 Protección de los datos personales

Responde a 4.1.5.1.19: protege la información sensible utilizando mecanismos seguros de almacenamiento y procesamiento.

R17 ordena resguardar el RUT y el teléfono. En los términos de la Ley 19.628 modificada por la Ley 21.719 son datos personales (no «sensibles», que la ley reserva para la salud, el origen étnico y otros), y el artículo 14 quinquies exige medidas de seguridad según el riesgo. Las medidas son:

- **Confidencialidad e integridad:** el RUT y el teléfono se cifran con Fernet (cryptography), que combina AES con un código de autenticación HMAC-SHA256. Un byte alterado del dato cifrado produce un error, nunca un RUT falso. La prueba lee el archivo de la base completo y no encuentra ningún RUT ni teléfono legible.
- **La clave, aparte:** se crea en el primer uso, en la carpeta de configuración del usuario (~/.config/viajes-aventura/), fuera del código, de la base y del proyecto. Una copia de la carpeta o de la base no se lleva la clave. Si se pierde con datos ya cifrados, el sistema se niega a crear otra (S-12).
- **Descifrado mínimo:** los datos van cifrados también en memoria y se descifran solo para enmascararlos. Iniciar sesión o listar reservas no descifra el RUT de nadie (minimización).
- **Nunca completos en pantalla:** se muestran como 12.***.***-5 y +56 9 **** 1234, también para su dueño; no aparecen en las trazas (__repr__) ni en los mensajes de error, que nombran el campo sin repetir el valor (RF-SEG-10 y RF-SEG-13).
- **Archivos:** la base, su diario y la clave quedan solo para su dueño (0600, máscara 077 al arrancar). Ni la base ni la clave viajan al repositorio, y el historial de git se revisó entero.
- **Deber de información** (artículo 14 ter): antes de registrarse, el cliente ve quién es el responsable, qué datos se piden y para qué, la base legal (la ejecución de la reserva que

solicita), que no se ceden a terceros, cuánto tiempo se conservan, cómo se protegen y cómo ejercer sus seis derechos con su plazo de respuesta; y debe aceptarlo.

- **Disponibilidad:** el respaldo de la base (RNF-FIA-03) responde a la otra mitad de P-14 («si se pierde, se pierde con todo»); la clave se respalda aparte.

Esto implica que la planilla que hoy circula por correo con los RUT en claro (P-14) deja de ser posible: lo que podría circular es una base cuyo contenido personal no se puede leer sin una clave que no está en ella.

5.9 Evaluación y mejora de la seguridad con apoyo de IA

Responde a 4.1.5.I.20: evalúa y optimiza la seguridad del sistema utilizando criterios técnicos y apoyo de IA.

La evaluación siguió un método propio de auditoría (fases de código, criptografía, cadena de suministro, pruebas e informe), y su detalle completo está en docs/AUDITORIA.md. Combinó herramientas automáticas con una revisión pedida a la IA (tabla 40).

Tabla 40. Resultado de la evaluación de seguridad

Fuente	Encontró	Resultado
Revisión con IA (Claude, en un agente nuevo sin contexto del proyecto, prompt CLARO)	17 hallazgos: 4 medios, 10 bajos y 3 informativos, con ubicación y categoría OWASP Top 10:2025	4 adoptados, 12 modificados y 1 descartado, cada uno con su fundamento
bandit (análisis estático)	6 consultas SQL armadas con texto (no explotables, pero contra RNF-SEG-03)	Las 6 corregidas; quedan 0
pip-audit (dependencias)	Ninguna vulnerabilidad conocida	0
Historial de git	Ningún archivo de base, clave ni secreto en ningún commit	0
SonarCloud (en cada envío)	Observaciones de mantenibilidad y un falso positivo de seguridad, a lo largo de los tres sprints	Quality Gate aprobado, 0 observaciones abiertas, 0 puntos críticos de seguridad
Corrección independiente con la rúbrica (agente sin contexto)	5 errores reales de código o de forma, y 25 brechas frente a la rúbrica y a las diapositivas del docente	Los 5 errores corregidos con su prueba; de las 30 recomendaciones, 20 adoptadas, 4 modificadas y 6 descartadas con su motivo

Validación crítica. Ninguna recomendación se aplicó sin verificarla. Las afirmaciones comprobables se probaron por experimento antes de actuar, y tres resultaron ser errores reales: la transacción que no cubría la consulta (H-08), el techo del total menor que el total posible, que habría dejado una reserva imposible de leer (H-09), y una opción de 5.000 dígitos que cerraba el programa (H-12). De las 12 modificaciones, cada una tiene un motivo: por ejemplo, la IA proponía rechazar una clave con permisos abiertos, y se optó por cerrarlos y seguir, porque rechazarla dejaría a la agencia sin poder atender (H-01). La IA tampoco vio una consecuencia de su propia recomendación H-08: con la transacción tomada desde el inicio, una conexión abierta dentro de otra se queda esperando. La encontró el recorrido del menú.

Mejoras y su verificación. Entre otras: el registro de auditoría de 23 acciones sin datos personales, la clave fuera del proyecto, el descifrado mínimo, la transacción desde la primera consulta, el plazo

de inactividad dentro de las acciones, el aviso de datos y la política de contraseñas comunes. Para comprobar que cada mejora quedó protegida, herramientas/mutaciones.py deshace cada corrección y rompe cada regla principal, una por vez, en una copia del código: las 27 mutaciones son detectadas por al menos una prueba, y el script corre en cada envío al repositorio. Los riesgos que no se corrigieron (desactivar cuentas, la enumeración de correos por el registro y el bloqueo progresivo) quedaron declarados con su motivo, lo que los mitiga hoy y cómo se resolverían.

Esto implica que la IA aportó 17 hallazgos con su ubicación en una sola revisión, pero el valor estuvo en la verificación: tres eran errores reales, doce necesitaban otra forma, y el error que impedía registrar al primer cliente no lo vio ella.

5.10 Pruebas y evidencia

Todo lo anterior se verifica en cada envío al repositorio con el workflow pruebas.yml, en seis combinaciones (Windows, macOS y Linux, con Python 3.12 y 3.14): instalación verificada por hash, autoverificación de las reglas, sesión completa del menú con los dos roles, comparación del diagrama con el código y la prueba por indicador de la rúbrica, con 31 afirmaciones. Las pruebas, a su vez, se probaron: de 46 mutaciones introducidas a propósito, todas fueron detectadas (una, recién después de corregir el comparador). El historial del repositorio muestra el trabajo en ramas feature/... integradas a main, cada uno con la historia de usuario en su mensaje.

6. Uso de inteligencia artificial

Responde a 4.1.4.I.16: utiliza, analiza y mejora críticamente el código generado mediante herramientas de IA.

Cómo se usó. El proyecto completo se construyó con Claude Code (modelo Claude Opus) como asistente: requerimientos, diagramas, código y pruebas. Lo dirigió el autor con instrucciones registradas íntegras, con fecha y categoría, en el registro de trabajo, y las decisiones de fondo fueron suyas: el alcance, la aprobación del modelo de clases, qué va al repositorio y qué se entrega. Además, en tres momentos se pidió una segunda opinión a un agente de Claude nuevo, que no conocía el proyecto: el diseño del modelo (con un prompt pobre y uno con el esquema CLARO), la auditoría de seguridad del código y una corrección del informe y del repositorio con la rúbrica, como la haría el docente. Las respuestas de las dos primeras están íntegras en docs/ia/ del repositorio.

Cómo se evaluó cada contribución. Ningún resultado se aceptó por venir de la IA. Cada uno pasó por una verificación: el verificador del catálogo, el comparador del diagrama, la autoverificación, el recorrido del menú, la prueba por indicador, bandit, pip-audit, SonarCloud o una prueba de mutación. Cuando la IA afirmaba un error, se comprobó por experimento antes de corregirlo. La tabla 41 muestra las contribuciones con más peso técnico, y la tabla 42, el conteo de las 93 registradas. El detalle de cada una está en docs/ANALISIS_IA.md.

Tabla 41. Contribuciones de la IA: qué se pidió, qué devolvió, qué se decidió y por qué

Qué se pidió	Qué devolvió la IA	Decisión	Fundamento técnico
--------------	--------------------	----------	--------------------

Diagrama de clases, con un prompt de una línea	Una clase AgenciaViajes con listas en memoria, el cupo como contador que se descuenta y el precio en float	Descartado	Ninguna frase del caso origina esa clase; un contador aparte es la causa de las 6 reservas sobre el cupo (R14 lo define como un cálculo); float contradice el total fijo de R13
El mismo diagrama, con el prompt CLARO y el caso adjunto	Una excepción ReglaNegocioError con el código de la regla, en un atributo público	Modificado	La idea se adoptó (el menú distingue el error esperado del inesperado), pero el atributo pasó a privado, porque el modelo declara todos sus atributos privados
El mismo	El margen como Decimal	Modificado	El peso no tiene decimales: un porcentaje entero y aritmética entera evitan el error de representación sin otra biblioteca
El mismo	Una clase Catalogo que reúne destinos y paquetes	Descartado	Ninguna frase del caso le da comportamiento propio: sus métodos ya estaban en Destino y Paquete
Implementar el registro y el inicio de sesión	Argon2id con los parámetros por omisión de la biblioteca	Modificado	Medido: 98 ms por verificación, bajo el mínimo de 0,1 s de RNF-REN-02; se subió a 4 pasadas (unos 120 ms)
Implementar las reservas	La reserva comprueba el cupo y guarda en una transacción BEGIN IMMEDIATE	Adoptado	Probado con dos conexiones que compiten por el último lugar: solo una reserva queda; la mutación que quita la transacción se detecta
El historial de reservas del cliente	Una consulta que une reserva y usuario y lee fila["id"]	Modificado (error corregido)	Las dos tablas tienen id: el historial armaba la reserva con el id equivocado y una cliente no podía anular la suya. Lo encontró el recorrido del menú
La clave de cifrado de los datos personales	La clave se creaba solo si la base no tenía usuarios	Modificado (error corregido)	En una instalación nueva el primer administrador existe antes que el primer cliente: el primer registro fallaba siempre. Lo encontró la prueba por indicador; la condición pasó a ser «no hay datos cifrados»
Auditar la seguridad del código	La clave de cifrado vivía en la carpeta del proyecto; rechazarla si sus permisos están abiertos	Modificado	La clave se movió a la carpeta del usuario; los permisos abiertos se cierran en vez de rechazarla, porque rechazarla dejaría a la agencia sin atender
La misma auditoría	La consulta previa a una escritura no entraba en la transacción	Modificado	Comprobado por experimento. Se corrigió con BEGIN IMMEDIATE en toda conexión; la IA no vio que eso dejaba esperando a una conexión abierta dentro de otra, y hubo que separarlas
La misma auditoría	El techo del total de una reserva era menor que el total posible	Adoptado	Comprobado por aritmética: una reserva grande se guardaba y después rompía el historial

La misma auditoría	Desactivar cuentas y suprimir los datos de un cliente; bloqueo progresivo	Descartado	Fuera del alcance del caso y con cambios de esquema; RF-SEG-03 fija 5 intentos y 5 minutos. Los dos quedaron declarados como riesgos aceptados
Corregir el informe y el repositorio con la rúbrica	«2.5» personas se registraba como 25	Adoptado	Confirmado: el menú borraba todos los puntos. Ahora el punto solo vale como separador de miles
La misma corrección	Un administrador armado sin contraseña tenía permisos	Adoptado	Confirmado: contradecía el criterio de RF-SEG-05. El permiso exige ahora una sesión iniciada (decisión 13)
La misma corrección	Separar los requerimientos con varias condiciones	Descartado	Cada uno es una sola regla de validación sobre un mismo dato; separarlos cambiaría siete identificadores ya trazados a casos de uso, historias y pruebas
La misma corrección	Las lecturas toman el candado de escritura: es ineficiente	Descartado	Es el costo medido de la atomicidad: con diez veces el volumen, la oferta se lista en 0,14 segundos

Tabla 42. Contribuciones de la IA y su clasificación

Fuente	Adoptados	Modificados	Descartados
Requerimientos, modelos y planificación (asistente)	2	3	3
Iteraciones independientes sobre el modelo de clases	0	5	11
Código y pruebas (asistente)	4	16	2
Auditoría de seguridad independiente	4	12	1
Corrección independiente del informe y el repositorio	20	4	6
Total (93)	30	40	23

De las 93 contribuciones, 30 se usaron como vinieron, siempre después de una prueba o una medición que las sostiene; 40 se modificaron y 23 se descartaron con su motivo. Los cinco errores reales que encontró la corrección independiente estaban en un trabajo que ya pasaba todas sus pruebas: ninguna los cubría porque nadie había pensado en esos casos, y ahora cada uno tiene la suya.

Esto implica que la IA aceleró el trabajo pero no decidió: lo que entró al sistema entró porque una prueba lo sostiene, y lo que no, quedó escrito con su motivo. La regla que dejó el proyecto es que la evidencia de que algo funciona es una prueba que falla cuando se rompe.

7. Trazabilidad

Responde a 4.1.2.1.8: valida la coherencia y trazabilidad entre requerimientos y modelos desarrollados.

La tabla 43 sigue cada uno de los 63 requerimientos, sin excepción: los 46 funcionales y los 17 no funcionales. Para cada uno muestra el caso de uso, la tarea o compuerta del proceso BPMN que lo ejecuta (cuando la tiene: 22 de 63), la clase y el método que lo implementan y la prueba concreta que lo comprueba. Los requerimientos de catálogo y de cuentas no tienen tarea BPMN porque los

procesos modelados son los tres que el caso describe con sus fallas o sus vacíos: reservar, armar un paquete y anular una reserva. Los no funcionales que valen para todo el sistema dicen «(transversal)» en lugar de un caso de uso.

La coherencia se verifica por programa en cada eslabón: `trazabilidad.py` (que revisa este informe y no viaja al repositorio) exige que todo RF esté en un caso de uso, que la figura y la tabla de casos de uso citen los mismos y que todo requerimiento tenga su historia en el backlog; `uml_vs_codigo.py` exige que cada método nombrado exista en el diagrama y en el código con los mismos parámetros; y la autoverificación, la prueba por indicador, la sesión del menú y las mutaciones corren en cada envío al repositorio. Ningún requerimiento queda sin método ni sin prueba.

Tabla 43. Matriz de trazabilidad: requerimiento, caso de uso, proceso, diseño y prueba

Requerimiento	Caso de uso	Tarea o compuerta BPMN	Clase y método	Se comprueba en
RF-DES-01	CU-09	(sin proceso BPMN)	<code>Destino.guardar()</code>	Prueba por indicador G.15 (CRUD de Destino) · Sesión del menú: registrar «Valle del Elqui»
RF-DES-02	CU-09	(sin proceso BPMN)	<code>Destino.guardar()</code> , <code>normalizar()</code> y <code>UNIQUE</code>	Autoverificación de destinos: «valle del elqui» y «Valle del Elquí» rechazados (R1) · Sesión del menú
RF-DES-03	CU-09	(sin proceso BPMN)	constructor de Destino y <code>CHECK</code>	Autoverificación de destinos: costo 0 y -1 y duración 0, también por SQL directo
RF-DES-04	CU-09	(sin proceso BPMN)	<code>Destino.editar()</code> y <code>cambiarCosto()</code>	Autoverificación de destinos: editar, y una edición fallida que no deja el objeto a medias · Prueba por indicador G.15
RF-DES-05	CU-09	(sin proceso BPMN)	<code>Destino.cambiarCosto()</code> (<code>fechaCosto</code>)	Autoverificación de destinos: costo a 130.000 con la fecha del cambio · Sesión del menú
RF-DES-06	CU-09	(sin proceso BPMN)	<code>Destino.eliminar()</code>	Autoverificación de destinos: un destino sin paquetes se elimina · Sesión del menú
RF-DES-07	CU-09	(sin proceso BPMN)	<code>Destino.eliminar()</code> (R8)	Autoverificación de destinos: Surire, que está en un paquete, queda no disponible
RF-DES-08	CU-09	(sin proceso BPMN)	<code>Destino.listar()</code>	Sesión del menú: listar el catálogo
RF-DES-09	CU-09	(sin proceso BPMN)	<code>Destino.reactivar()</code>	Autoverificación de destinos: Surire se vuelve a ofrecer
RF-DES-10	CU-09	(sin proceso BPMN)	<code>Destino.listar(soloDisponibles)</code>	Autoverificación de destinos: el filtro excluye al no disponible · Sesión del menú

RF-PAQ-01	CU-10	Ingresar nombre, fechas, cupo, margen y destinos · Guardar el paquete en borrador	Paquete.guardar()	Autoverificación de paquetes: «Altiplano y estrellas» en borrador · Prueba por indicador G.15
RF-PAQ-02	CU-10	Validar los datos y los destinos · compuerta «¿2 a 5 destinos disponibles...?»	constructor de Paquete y Paquete.guardar()	Autoverificación de paquetes: 1 y 6 destinos, uno repetido y uno no disponible (R3, R8) · Sesión del menú
RF-PAQ-03	CU-10	Validar los datos y los destinos	constructor de Paquete y CHECK	Autoverificación de paquetes: regreso igual a la salida, cupo 0 y margen -5 % · Prueba por indicador G.14 (SQL directo)
RF-PAQ-04	CU-10	Calcular el precio por persona	Paquete.calcularPrecio()	Autoverificación de paquetes: 310.000 + 120.000 con 20 % da 516.000 · Sesión del menú: \$528.000
RF-PAQ-05	CU-11	compuerta «¿La salida todavía no llega...?» · Fijar el precio y publicar (R7)	Paquete.publicar()	Autoverificación de paquetes: el precio publicado no cambia al subir un costo
RF-PAQ-06	CU-03	Mostrar paquetes disponibles con precio y cupo	Paquete.listarDisponibles() y cupoDisponible()	Autoverificación de paquetes: cupo 12, 10 vigentes y 2 anuladas, quedan 2
RF-PAQ-07	CU-03	Mostrar paquetes disponibles con precio y cupo	Paquete.listarDisponibles(hoy) y estado(hoy)	Autoverificación de paquetes: el día de la salida no está en la oferta y figura vencido
RF-PAQ-08	CU-10	(sin proceso BPMN)	Paquete.editar(), reemplazarDestinos() y cambiarCupo()	Autoverificación de paquetes: borrador editable; publicado, solo el cupo; cupo 9 con 10 reservadas, rechazado
RF-PAQ-09	CU-10	(sin proceso BPMN)	Paquete.eliminar()	Autoverificación de paquetes: con reservas no se elimina, sin reservas sí · Prueba por indicador G.15
RF-PAQ-10	CU-10	(sin proceso BPMN)	Paquete.listarTodos() y estado()	Autoverificación de paquetes: el cliente no puede listarlos · Sesión del menú: listar todos
RF-PAQ-11	CU-10	Ingresar nombre, fechas, cupo, margen y destinos	constructor de Paquete (margen = 20)	Autoverificación de paquetes: margen por omisión · Sesión del menú: Enter deja 20 %

RF-RES-01	CU-01	(sin proceso BPMN)	Cliente.registrar()	Autoverificación de cuentas: registro de Carolina · Prueba por indicador G.15 · Sesión del menú
RF-RES-02	CU-01	(sin proceso BPMN)	Cliente.registrar() y UNIQUE del correo (R9)	Autoverificación de cuentas: «Carolina@Correo.cl» repetido
RF-RES-03	CU-01	(sin proceso BPMN)	constructor de Cliente y validarRut	Autoverificación de cuentas: RUT, correo y teléfono inválidos · Sesión del menú: dígito verificador malo
RF-RES-04	CU-04	Elegir paquete e indicar personas · Guardar la reserva con su total fijo, en una transacción	Reserva.reservar()	Autoverificación de reservas: 2 personas, vigente · Prueba por indicador G.15 · Sesión del menú
RF-RES-05	CU-04	compuerta «¿Las personas superan el cupo disponible?» · Informar los cupos que quedan	Reserva.reservar() (R14, R16)	Autoverificación de reservas: 0 personas y 3 con 2 lugares («quedan 2») · Sesión del menú: «2.5» rechazado
RF-RES-06	CU-04	compuerta «¿La fecha de salida ya llegó?» · Informar fecha vencida	Reserva.reservar(hoy) (R15)	Autoverificación de reservas: reservar el mismo día de la salida
RF-RES-07	CU-04	Calcular el total y advertir si ya tiene una reserva vigente	Reserva.reservar() y obtenerTotal()	Autoverificación de reservas: 1.032.000 que no cambia al subir un costo
RF-RES-08	CU-05	Anular: Mostrar las reservas propias, numeradas	Cliente.historial()	Autoverificación de reservas: cada cliente ve las suyas, con la anulada, y ninguna ajena
RF-RES-09	CU-06	Anular: compuerta «¿Es suya, está vigente y su salida no llegó?» · Marcarla anulada	Reserva.anular()	Autoverificación de reservas: la propia sí, la ajena no, dos veces no · Sesión del menú
RF-RES-10	CU-04	Calcular el total y advertir si ya tiene una reserva vigente · compuerta «¿Confirma?»	Cliente.tieneReservaVi gente()	Autoverificación de reservas · Sesión del menú: advertencia y respuesta «no»
RF-RES-11	CU-12	(sin proceso BPMN)	Reserva.listarPorPaque te()	Autoverificación de reservas: 5 reservas con nombre y correo, sin RUT · Sesión del menú
RF-RES-12	CU-07	(sin proceso BPMN)	Cliente.actualizarCont acto()	Autoverificación de cuentas: nombre y teléfono nuevos · Prueba por indicador G.15 · Sesión del menú
RF-SEG-01	CU-02	Iniciar sesión	Usuario.autenticar()	Autoverificación de cuentas · Prueba por indicador G.18 · Sesión del menú

RF-SEG-02	CU-02	Iniciar sesión	Usuario.autenticar() con hash señuelo	Prueba por indicador G.18: mismo mensaje y misma demora · Sesión del menú: dos fallos con el mismo mensaje
RF-SEG-03	CU-02	Iniciar sesión	Usuario.autenticar() (bloqueo en la base)	Autoverificación de cuentas: 5 fallos bloquean aun con la clave correcta · Prueba por indicador G.18
RF-SEG-04	CU-01, CU-05 (sin proceso BPMN)		Usuario.validarClave()	Autoverificación de cuentas: 11 caracteres, igual al correo y comunes · Sesión del menú: «corta»
RF-SEG-05	CU-02	(sin proceso BPMN)	Usuario.puede(), tieneSesion() y autorizar()	Autoverificación de destinos: cliente y administrador sin sesión, rechazados · Prueba por indicador G.13
RF-SEG-06	CU-13	(sin proceso BPMN)	Administrador.crearPri mero()	Autoverificación de cuentas: la primera cuenta, una sola vez · Sesión del menú: primer uso
RF-SEG-07	CU-13	(sin proceso BPMN)	Administrador.crearAdm inistrador()	Autoverificación de cuentas: el socio tiene permisos recién con su sesión · Sesión del menú
RF-SEG-08	CU-02	(sin proceso BPMN)	menú: cerrar sesión	Sesión del menú: «Sesión cerrada.»
RF-SEG-09	CU-02	(sin proceso BPMN)	menú: plazo en cada espera de un dato	Sesión del menú con el reloj adelantado: en el menú y dentro de una acción
RF-SEG-10	CU-05, CU-12 (sin proceso BPMN)		Cliente.rutEnmascarado () y telefonoEnmascarado()	Autoverificación de cuentas: 12 ***-***-5 y +56 9 **** 5678 · Prueba por indicador I.19
RF-SEG-11	CU-08	(sin proceso BPMN)	Usuario.cambiarClave()	Autoverificación de cuentas y de auditoría: actual equivocada, igual a la actual y sesión vieja · Prueba por indicador G.18
RF-SEG-12	CU-01	(sin proceso BPMN)	Cliente.registrar()	Autoverificación de cuentas: el registro crea un Cliente · Sesión del menú
RF-SEG-13	CU-01	(sin proceso BPMN)	validarRut() y validarTelefono() (mensajes)	Autoverificación de cuentas: ningún mensaje repite el RUT ni el teléfono · Prueba por indicador I.19
RNF-FIA-01	(transversal)	(sin proceso BPMN)	esquema: 17 CHECK, UNIQUE y claves foráneas	Prueba por indicador G.14: seis datos inválidos rechazados por SQL directo

RNF-FIA-02	CU-04	Guardar la reserva con su total fijo, en una transacción	Reserva.reservar() y conectar() (BEGIN IMMEDIATE)	Autoverificación de reservas: dos conexiones por el último lugar, una sola reserva
RNF-FIA-03	(transversal)	(sin proceso BPMN)	Administrador.respalda rBase()	Autoverificación de destinos: el respaldo abre y tiene los mismos registros · Sesión del menú
RNF-LEG-01	CU-01	(sin proceso BPMN)	aviso de datos del registro y Cliente	Sesión del menú: aviso del art. 14 ter · tabla de datos de la sección 1.2
RNF-MAN-01	(transversal)	(sin proceso BPMN)	herramientas/uml_vs_codigo.py	Workflow: 8 clases, 82 miembros, 0 diferencias
RNF-MAN-02	(transversal)	(sin proceso BPMN)	autoverificación y herramientas/mutaciones.py	Workflow: 27 de 27 mutaciones detectadas
RNF-POR-01	(transversal)	(sin proceso BPMN)	requirements.txt con los hashes de todas las plataformas	Workflow: Windows, macOS y Linux con Python 3.12 y 3.14
RNF-REN-01	CU-03, CU-04	Mostrar paquetes disponibles con precio y cupo	Paquete.listarDisponibles() y Cliente.historial()	Prueba por indicador RNF-REN: 0,14 s y 0,02 s con 10 veces el volumen
RNF-REN-02	CU-02	Iniciar sesión	Argon2id con 64 MiB y 4 pasadas	Prueba por indicador G.17: unos 120 ms en el equipo de desarrollo
RNF-SEG-01	CU-01, CU-02, CU-03	Iniciar sesión	Usuario.autenticar() con Argon2id (argon2-cffi)	Prueba por indicador G.17: todo hash es \$argon2id\$ y ninguna sal se repite
RNF-SEG-02	CU-01, CU-02	(sin proceso BPMN)	Cliente: rut y telefono {cifrado} (Fernet)	Prueba por indicador I.19: ningún RUT legible en la base; un byte alterado da error
RNF-SEG-03	(transversal)	(sin proceso BPMN)	texto(), entero() y consultas SQL literales	bandit: 0 B608 · Autoverificación: tipos y rangos
RNF-SEG-04	(transversal)	(sin proceso BPMN)	crear_tablas() y cifrador()	Autoverificación: permisos 0600 de la base y la clave · Prueba por indicador I.19
RNF-SEG-05	(transversal)	(sin proceso BPMN)	menú: atender() y main()	Sesión del menú: 0 Traceback, también con una opción de 5.000 dígitos
RNF-USA-01	(transversal)	(sin proceso BPMN)	menú: leer() y Cancelado	Sesión del menú: «x» cancela sin guardar
RNF-USA-02	(transversal)	(sin proceso BPMN)	pesos() y fechas día-mes-año	Sesión del menú: \$1.056.000 y 02-11-2026

RNF-USA-03	CU-02	(sin proceso BPMN)	menú: opciones_de() con puede()	Sesión del menú: el cliente no ve destinos, paquetes ni cuentas
------------	-------	--------------------	--------------------------------------	---

Esto implica que cualquier requerimiento se puede seguir hasta el código que lo cumple y la prueba que lo demuestra, y que si alguno se rompe, la prueba falla en el siguiente envío.

8. Conclusiones

El problema quedó cubierto. Las fallas que la agencia midió en su temporada tienen cada una su regla y su prueba: las 6 reservas sobre el cupo no pueden repetirse porque el cupo se calcula y la reserva se guarda en una transacción que no deja pasar a otra al mismo tiempo; los 4 paquetes cobrados a otro precio, porque el precio queda fijo al publicar; y la venta de un paquete vencido, porque la fecha se compara al reservar y el paquete sale solo de la oferta. Los 63 requerimientos (46 funcionales y 17 no funcionales) están implementados y cada uno tiene la prueba que lo comprueba, sin diferencias entre el diagrama y el código, y la aplicación funciona en Windows, macOS y Linux.

Lo que queda para una versión 2 son los ocho Won't del capítulo 2 (el pago, la facturación electrónica, el envío de correos, la aplicación para teléfonos y la integración con terceros, entre otros) y tres riesgos declarados en la auditoría: desactivar la cuenta de un socio que deja la agencia y suprimir los datos de un cliente (la Ley 21.719 rige desde el 1 de diciembre de 2026), verificar el correo al registrarse y un bloqueo progresivo de las cuentas.

Lo que se haría distinto. El primer sprint empezó un día tarde porque la preparación se extendió, y los tres sprints tomaron 5,4 horas contra 26 estimadas: la estimación supuso escribir el código a mano y no consideró la velocidad del asistente, y la próxima debería medirse con la velocidad ya registrada (79 puntos en 5,4 horas). Y las pruebas se escribirían desde el inicio recorriendo el sistema real: los dos errores reales del desarrollo (el historial con el id equivocado y el primer cliente que no podía registrarse) se escaparon a las pruebas que preparaban el escenario por fuera.

Sobre la IA. El criterio para usarla vino de las pruebas: de sus 93 contribuciones, 23 se descartaron y 40 se modificaron. Aportó una auditoría de 17 hallazgos, alternativas de diseño que confirmaron o mejoraron el modelo y una corrección con la rúbrica que encontró cinco errores reales; el error que impedía registrar al primer cliente, en cambio, lo encontró una prueba que recorría el sistema como lo usa una persona.

Referencias

- Biblioteca del Congreso Nacional de Chile. (2024). *Ley N.° 21.719, que regula la protección y el tratamiento de los datos personales y crea la Agencia de Protección de Datos Personales*. Biblioteca del Congreso Nacional de Chile. <https://www.bcn.cl/leychile/navegar?idNorma=1209272>
- Biblioteca del Congreso Nacional de Chile. (1999). *Ley N.° 19.628, sobre protección de la vida privada*. <https://www.bcn.cl/leychile/navegar?idNorma=141599>

- Biryukov, A., Dinu, D., Khovratovich, D. y Josefsson, S. (2021). *Argon2 memory-hard function for password hashing and proof-of-work applications* (RFC 9106). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc9106>
- International Organization for Standardization. (2013). *ISO/IEC 19510:2013. Information technology: Object Management Group Business Process Model and Notation*. ISO.
- International Organization for Standardization. (2023). *ISO/IEC 25010:2023. Systems and software engineering: Systems and software Quality Requirements and Evaluation (SQuaRE). Product quality model*. ISO.
- Object Management Group. (2017). *Unified Modeling Language (UML), versión 2.5.1*. <https://www.omg.org/spec/UML/2.5.1>
- OWASP Foundation. (2025). *OWASP Top 10:2025*. <https://owasp.org/Top10/2025/>
- OWASP Foundation. (s. f.). *Password Storage Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- Python Cryptographic Authority. (2026). *cryptography: Fernet (symmetric encryption)* (versión 50.0.2). <https://cryptography.io/en/latest/fernet/>
- Python Software Foundation. (2026). *sqlite3: DB-API 2.0 interface for SQLite databases*. <https://docs.python.org/3/library/sqlite3.html>
- Schlawack, H. (2025). *argon2-cffi: Argon2 for Python* (versión 25.1.0). <https://argon2-cffi.readthedocs.io/>
- Schnettler, R. (2026). *Caso Viajes Aventura y guía de la Evaluación Sumativa 4* [Material de la asignatura TI3021 Programación Orientada a Objeto Seguro]. INACAP.
- Schnettler, R. (2026). *Del encargo al informe técnico* [Diapositivas de la asignatura TI3021, bloque 6]. INACAP.
- Schwaber, K. y Sutherland, J. (2020). *La Guía Scrum*. <https://scrumguides.org/>